

Prob_ODE_test

January 30, 2022

```
[1]: # Make inline plots vector graphics instead of raster graphics
%matplotlib inline
from IPython.display import set_matplotlib_formats

set_matplotlib_formats("pdf", "svg")

# Plotting
import matplotlib.pyplot as plt
```

```
[2]: import numpy as np
from probnum.diffeq import probsolve_ivp
```

1 2-D damped oscillator (Linear)

```
[3]: def f(t, y):
    y1, y2 = y
    return np.array([-0.1*y1+2*y2, -2*y1-0.1*y2])

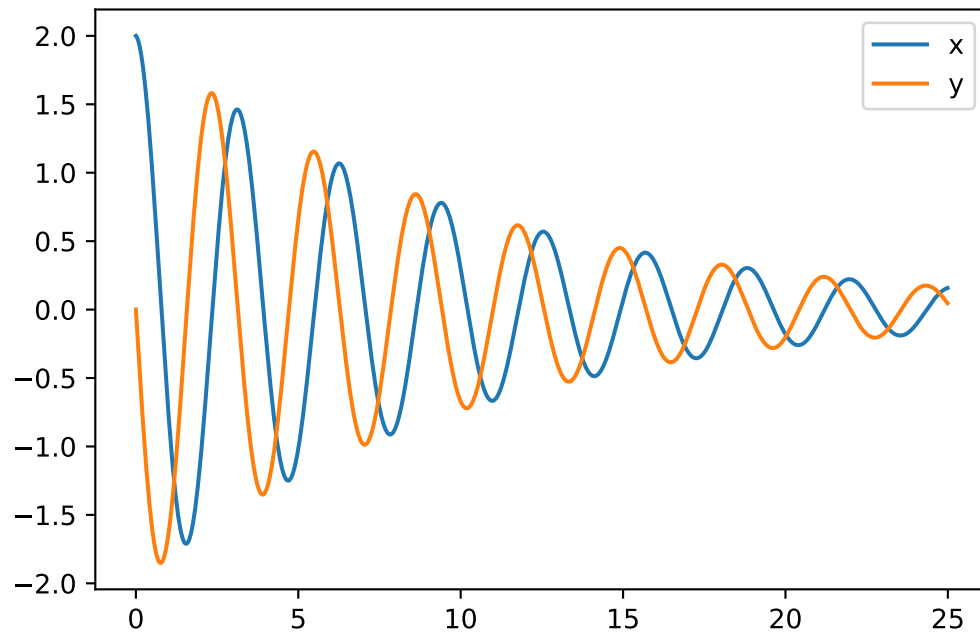
def df(t, y):
    y1, y2 = y
    return np.array([[ -0.1, 2], [-2, -0.1]])

t0 = 0.0
tmax = 25.0
y0 = np.array([2, 0])
```

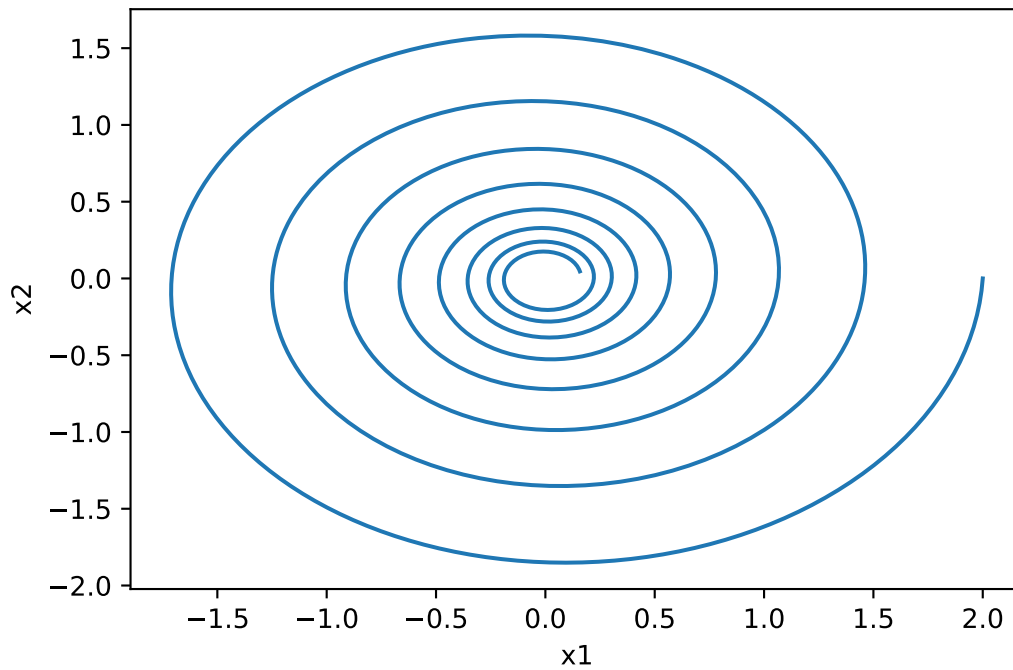
```
[4]: # IBM(1) with EK0, solving an IVP with adaptive steps works.
sol = probsolve_ivp(f, t0, tmax, y0, df=df, algo_order=4, method="EK1")
dt=0.01
evalgrid = np.arange(0, 25, dt)
means = sol(evalgrid).mean

plt.plot(evalgrid, means[:, 0], label="x")
```

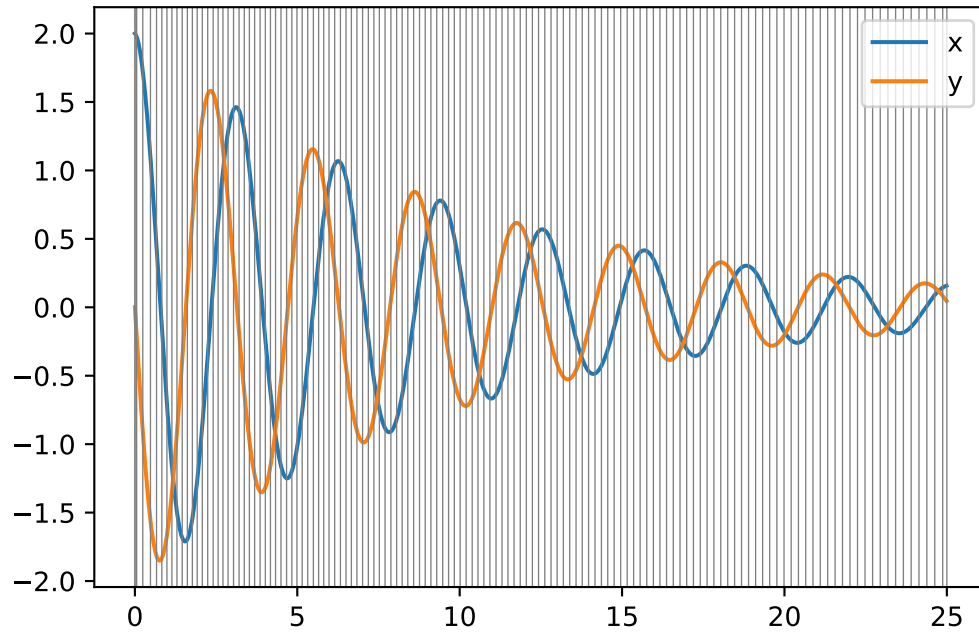
```
plt.plot(evalgrid, means[:, 1], label="y")
plt.legend(loc="upper right")
plt.show()
```



```
[5]: plt.plot(means[:, 0], means[:, 1])
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()
```



```
[6]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```



```
[7]: algo_orders = [2, 3]
filters = ["EK0", "EK1"]

fig, ax = plt.subplots(
    nrows=len(algo_orders),
    ncols=len(filters),
    sharex=True,
    sharey=True,
    figsize=(12, 12),
)

evalgrid = np.linspace(t0, tmax, 100)

for idx in range(len(algo_orders)): # algo_orders in rows
    for jdx in range(len(filters)): # filters in cols
        sol = probsolve_ivp(
            f,
            t0,
            tmax,
            y0,
            df=df,
            algo_order=algo_orders[idx],
            method=filters[jdx],
        )
```

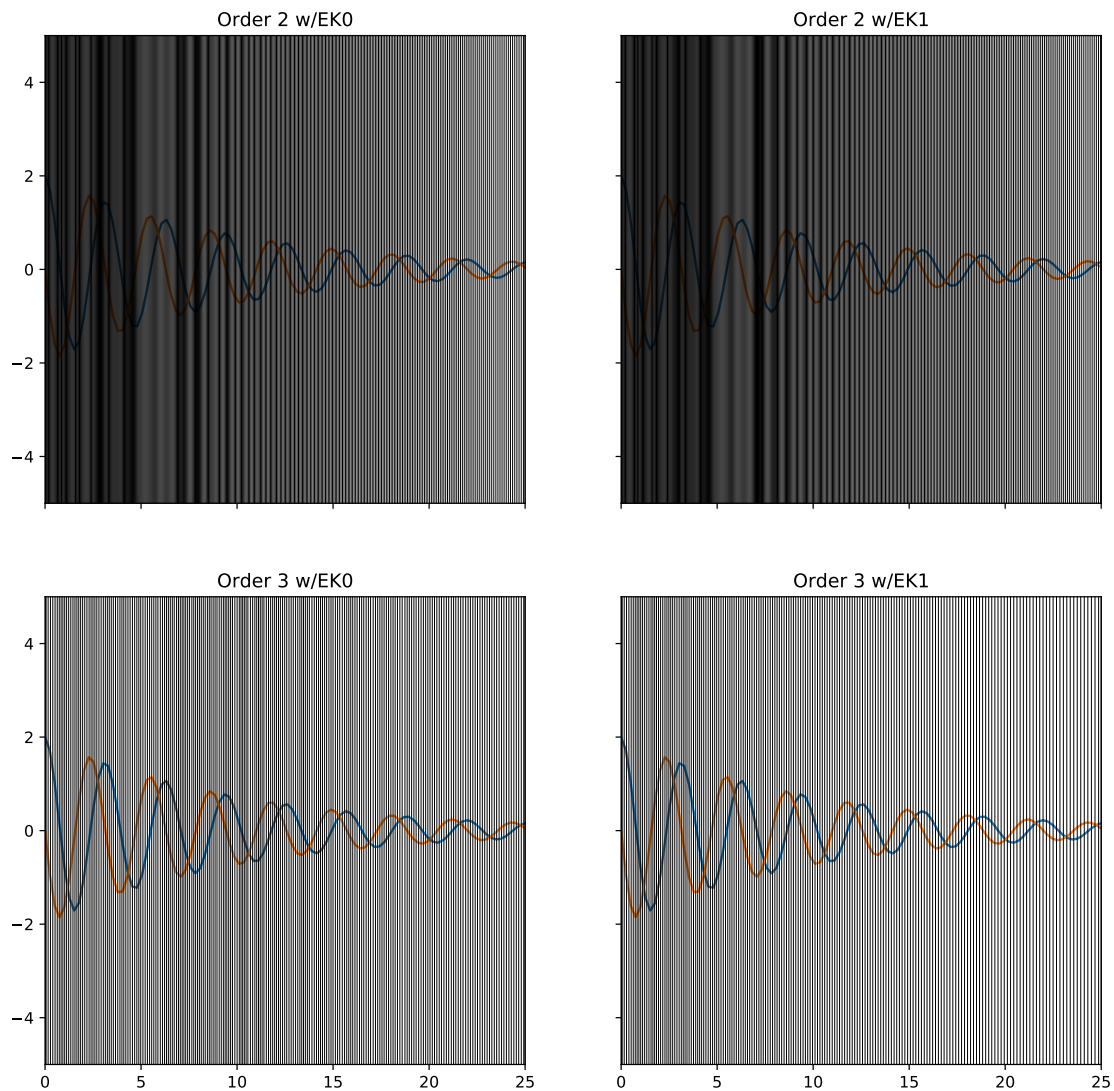
```

solution = sol(evalgrid)
ts, means, stds = evalgrid, solution.mean, solution.std
ax[idx][jdx].plot(ts, means)

for t in sol.locations:
    ax[idx][jdx].axvline(t, linewidth=0.2, color="black")
ax[idx][jdx].set_title(f"Order {algo_orders[idx]} w/{filters[jdx]}")
ax[idx][jdx].set_xlim((t0, tmax))
ax[idx][jdx].set_ylim((-5, 5))

plt.show()

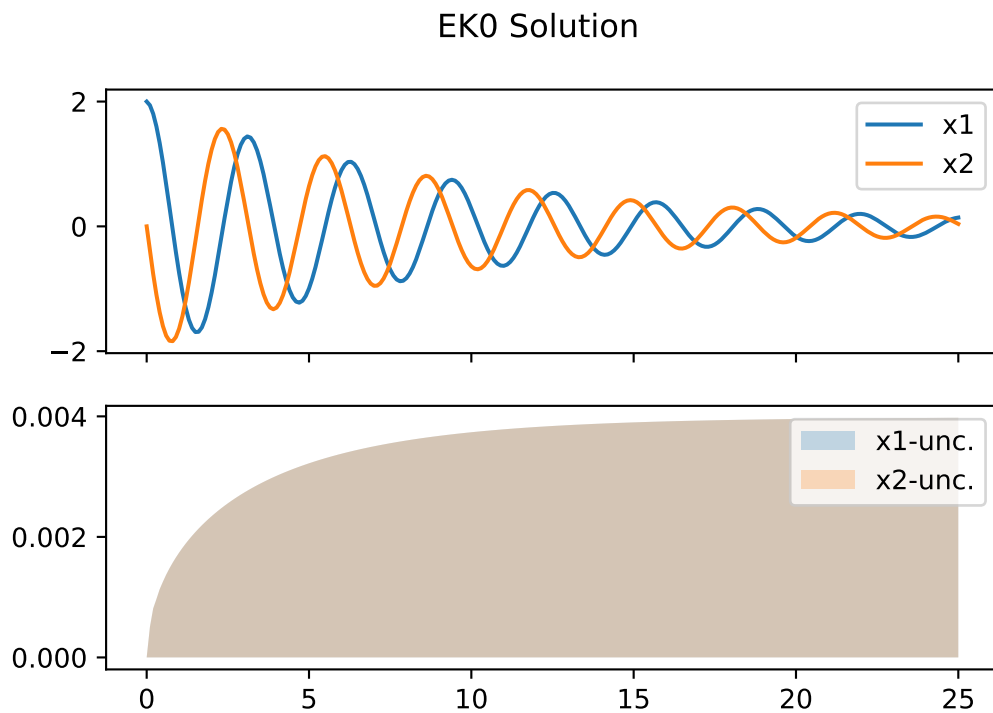
```



```
[8]: sol = probsolve_ivp(
    f, t0, tmax, y0, df=df, step=0.1, adaptive=False, diffusion_model="dynamic"
)
```

```
[9]: means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.legend()
fig.suptitle("EK0 Solution")
plt.show()
```



```
[10]: sol = probsolve_ivp(
    f,
    t0,
    tmax,
    y0,
    df=df,
    step=0.5,
```

```

    algo_order=1,
    adaptive=False,
    diffusion_model="dynamic",
)

```

```

[11]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

```

```

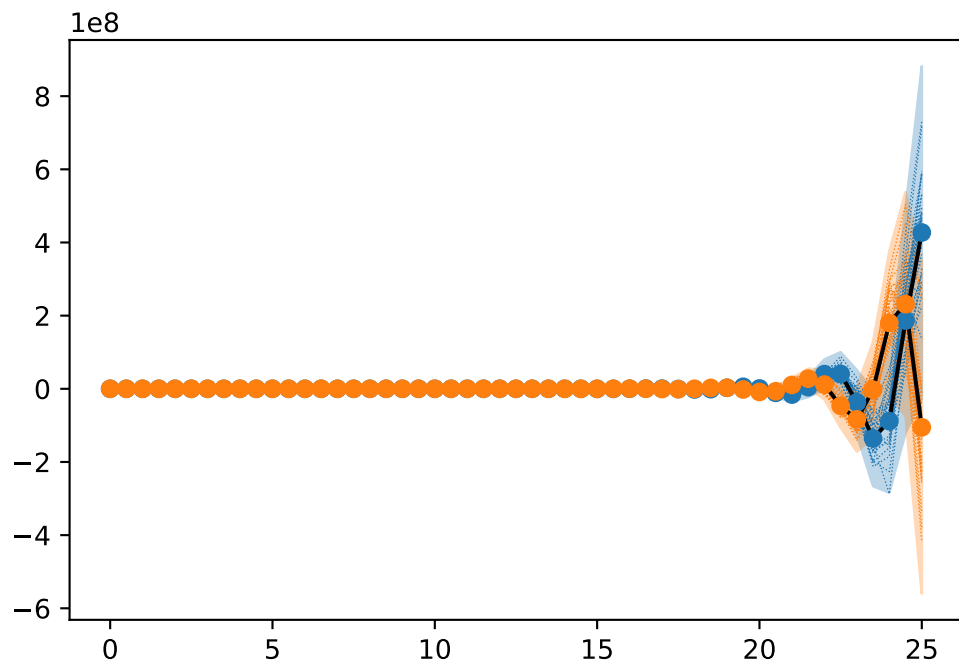
[12]: fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[::5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)

ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EKO Solution")

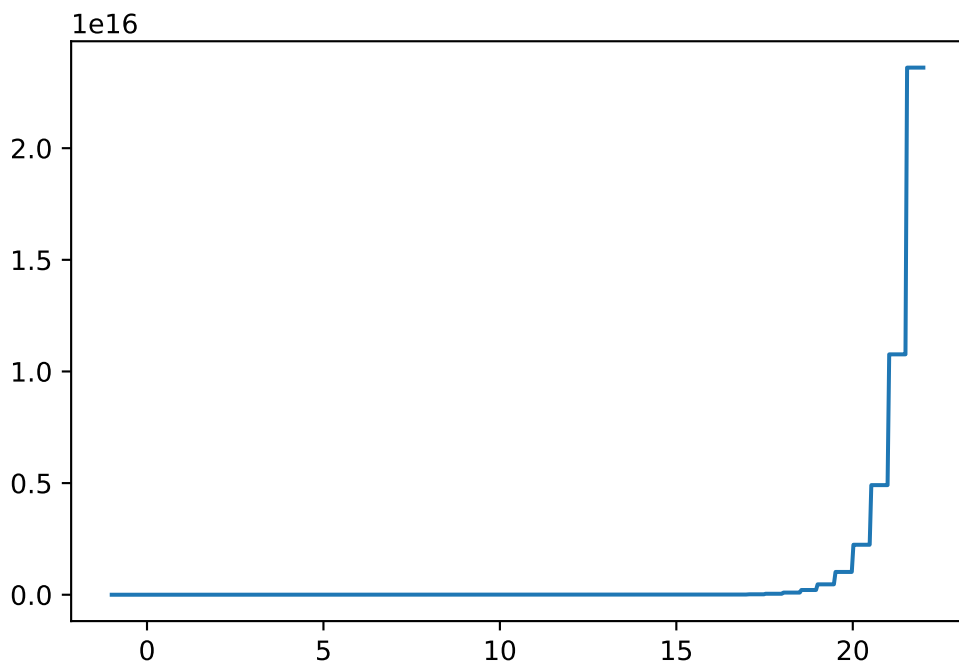
plt.show()

```

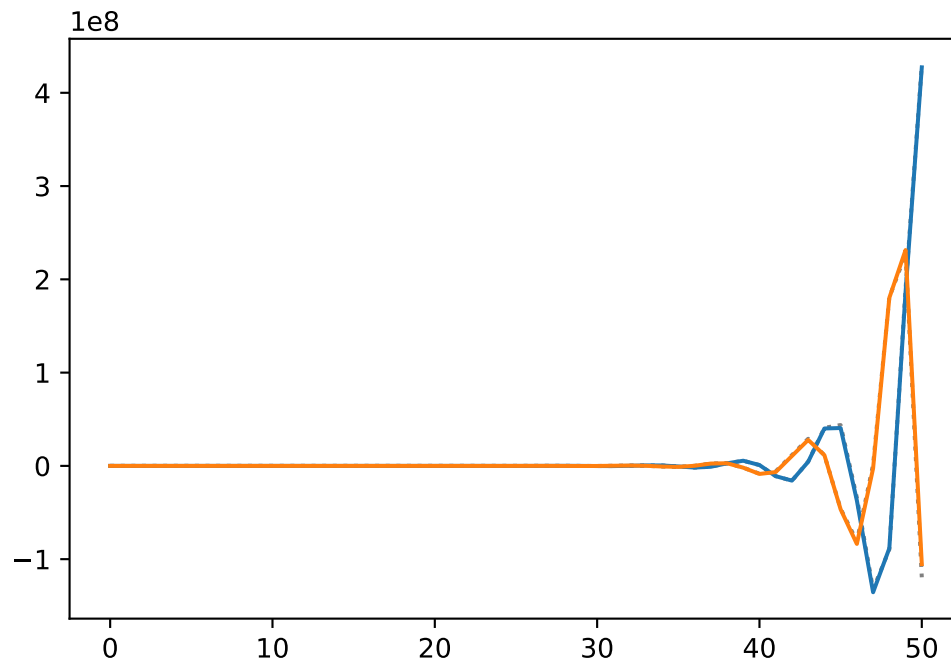
Samples From an EK0 Solution



```
[13]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[14]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

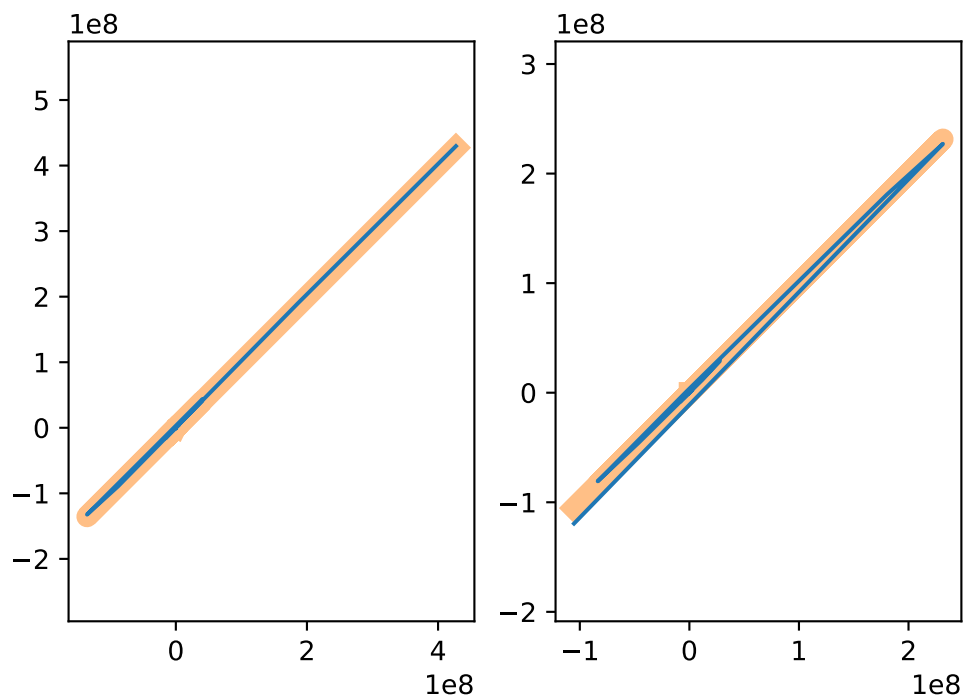


```
[15]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2) = plt.subplots(ncols=2)

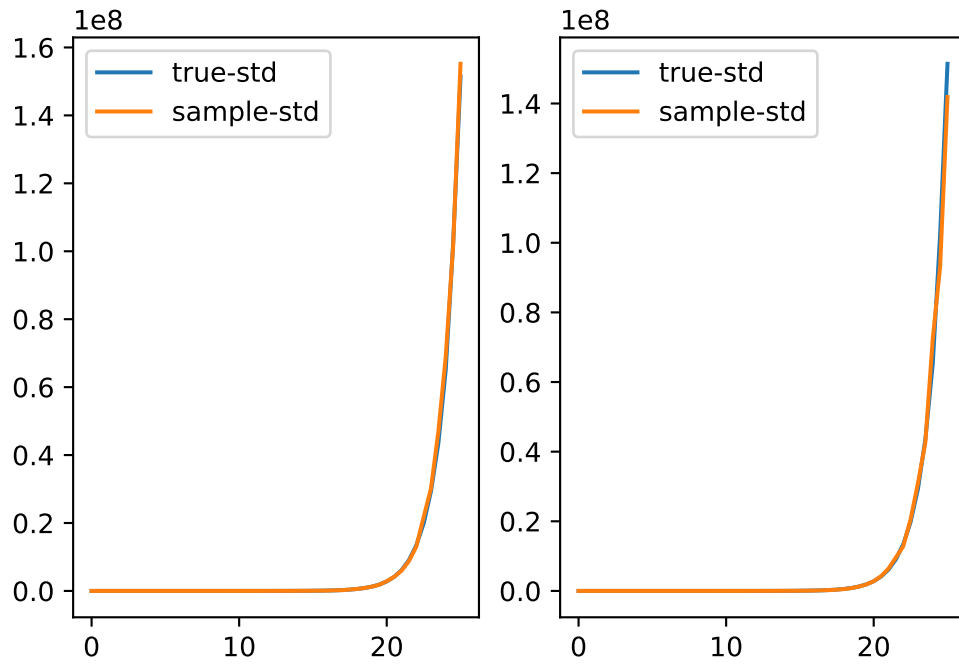
ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



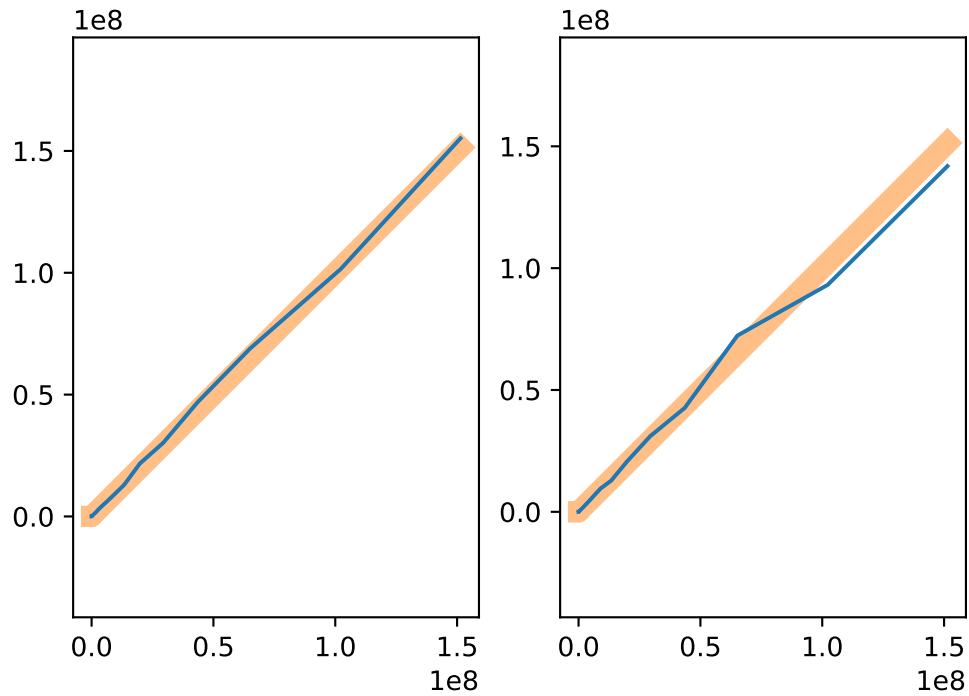
```
[16]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()
```



```
[17]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



[]:

[]:

[]:

```
[18]: from probnum import diffeq, filtsmooth, randvars, randprocs, problems
import numpy as np

import matplotlib.pyplot as plt
```

```
[19]: def f(t, y):
    y1, y2 = y
    return np.array([-0.1*y1+2*y2, -2*y1-0.1*y2])

def df(t, y):
    y1, y2 = y
    return np.array([[ -0.1, 2], [-2, -0.1]])

t0 = 0.0
tmax = 25.0
y0 = np.array([2, 0])
```

```
ivp = problems.InitialValueProblem(t0=t0, tmax=tmax, y0=y0, f=f, df=df)
```

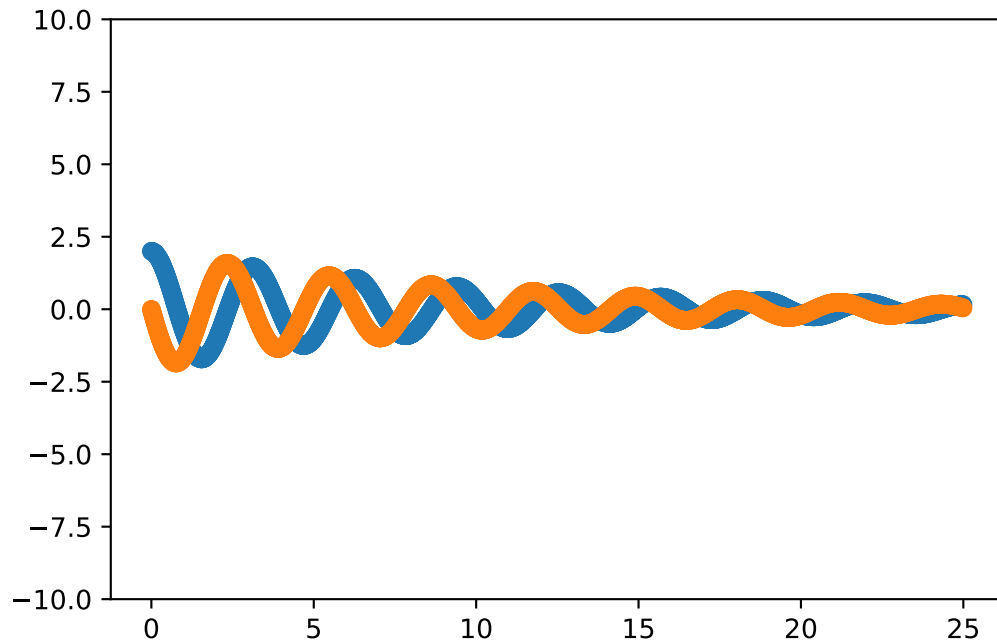
```
[20]: prior_process = randprocs.markov.integrator.IntegratedWienerProcess(
        initarg=ivp.t0,
        num_derivatives=4,
        wiener_process_dimension=ivp.dimension,
        forward_implementation="sqrt",
        backward_implementation="sqrt",
    )
```

```
[21]: diffmodel = randprocs.markov.continuous.PiecewiseConstantDiffusion(t0=t0)
rk_init = diffeq.odefilter.initialization_routines.RungeKuttaInitialization()
ode_residual = diffeq.odefilter.information_operators.ODEResidual(4, ivp.
    ↪dimension)
ek1 = diffeq.odefilter.approx_strategies.EK1()
firststep = diffeq.stepsize.propose_firststep(ivp)
steprule = diffeq.stepsize.AdaptiveSteps(firststep=firststep, atol=1e-3, ↪
    ↪rtol=1e-5)

solver = diffeq.odefilter.ODEFilter(
    steprule,
    prior_process=prior_process,
    information_operator=ode_residual,
    approx_strategy=ek1,
    initialization_routine=rk_init,
    diffusion_model=diffmodel,
    with_smoothing=True,
)
```

```
[22]: odesol = solver.solve(ivp=ivp)
evalgrid = np.arange(ivp.t0, ivp.tmax, step=0.01)
sol = odesol(evalgrid)

plt.plot(evalgrid, sol.mean, "o-", linewidth=1)
plt.ylim((-10, 10))
plt.show()
```



[]:

[]:

2 2-D damped oscillator (Nonlinear)

```
[23]: def f(t, y):
        y1, y2 = y
        return np.array([-0.1*y1**3+2*y2**3, -2*y1**3-0.1*y2**3])

        def df(t, y):
            y1, y2 = y
            return np.array([[-0.3*y1**2, 2*y2**2], [-2*y1**2, -0.1*y2**2]])

        t0 = 0.0
        tmax = 25.0
        y0 = np.array([2, 0])
        ivp = problems.InitialValueProblem(t0=t0, tmax=tmax, y0=y0, f=f, df=df)
```

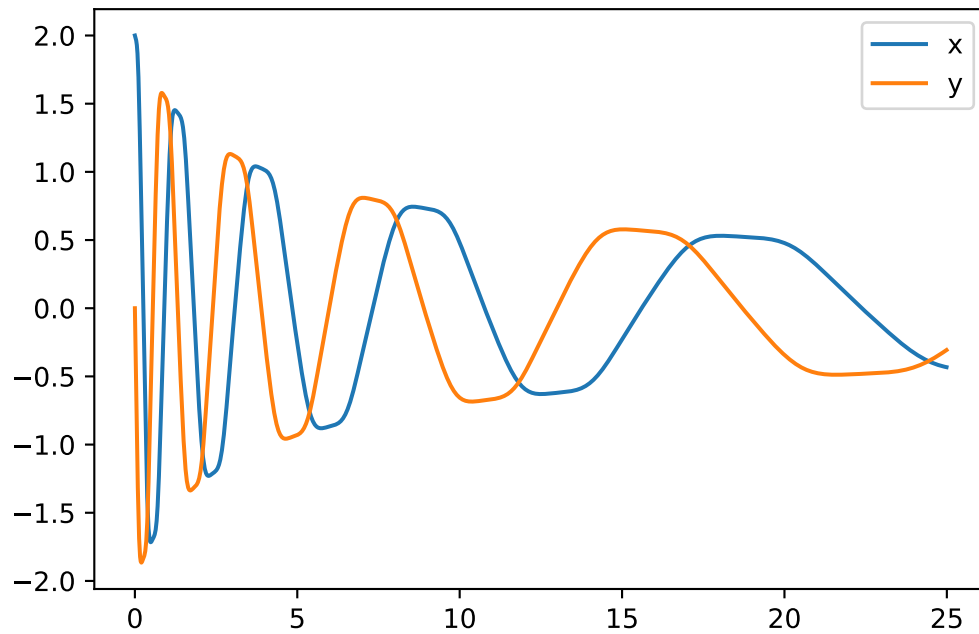
```
[24]: # IBM(1) with EK0, solving an IVP with adaptive steps works.
        sol = probsolve_ivp(t0=t0, tmax=tmax, y0=y0, f=f, df=df)
```

```

dt=0.01
evalgrid = np.arange(0, 25, dt)
means = sol(evalgrid).mean

plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.legend(loc="upper right")
plt.show()

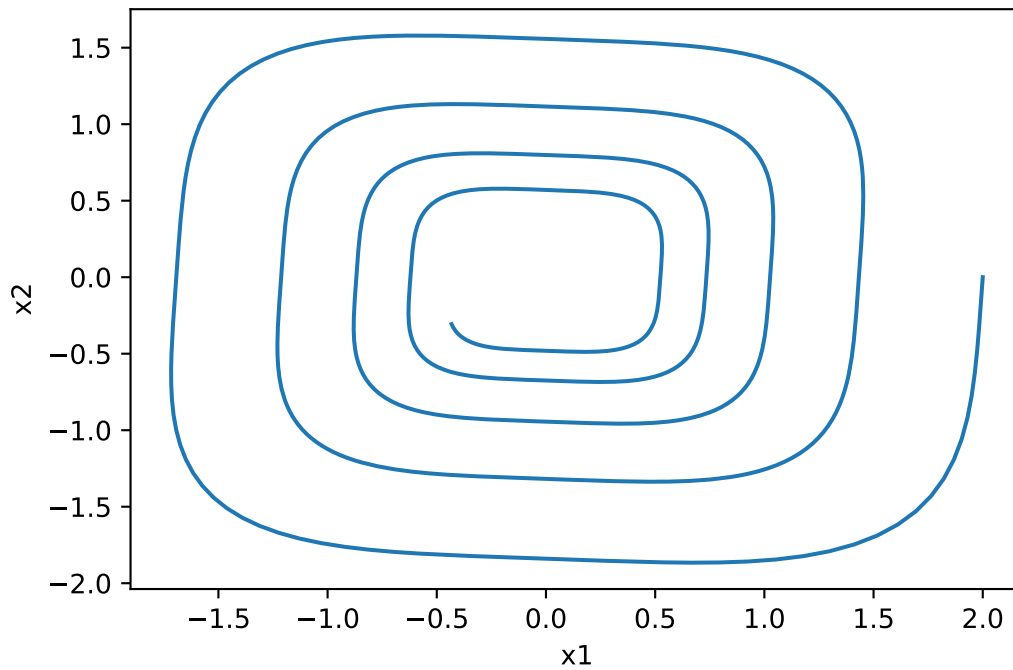
```



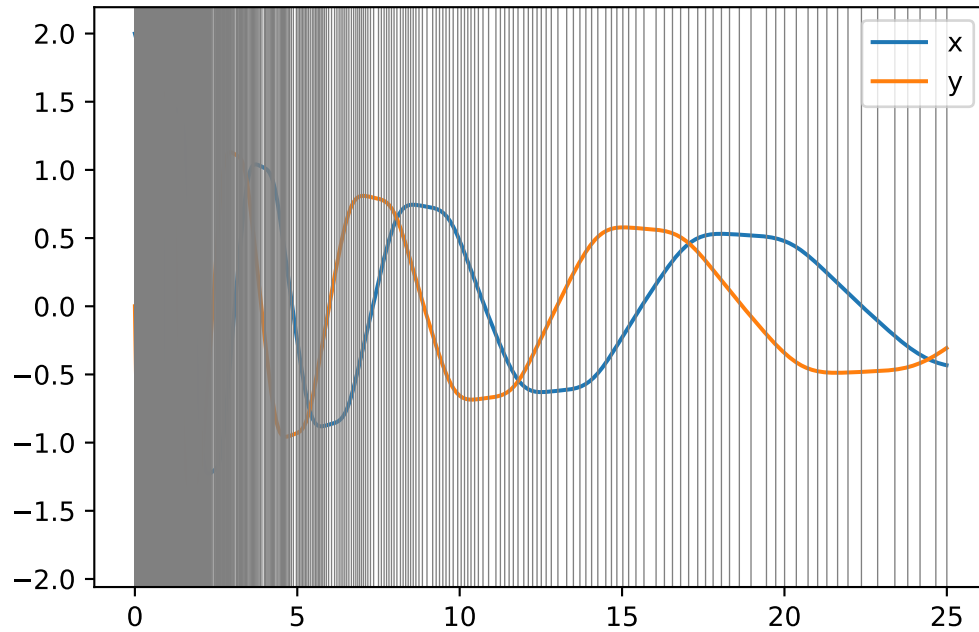
```

[25]: plt.plot(means[:, 0], means[:, 1])
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()

```



```
[26]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```



```
[27]: algo_orders = [2, 3]
filters = ["EK0", "EK1"]

fig, ax = plt.subplots(
    nrows=len(algo_orders),
    ncols=len(filters),
    sharex=True,
    sharey=True,
    figsize=(12, 12),
)

evalgrid = np.linspace(t0, tmax, 100)

for idx in range(len(algo_orders)): # algo_orders in rows
    for jdx in range(len(filters)): # filters in cols
        sol = probsolve_ivp(
            f,
            t0,
            tmax,
            y0,
            df=df,
            algo_order=algo_orders[idx],
            method=filters[jdx],
        )
```

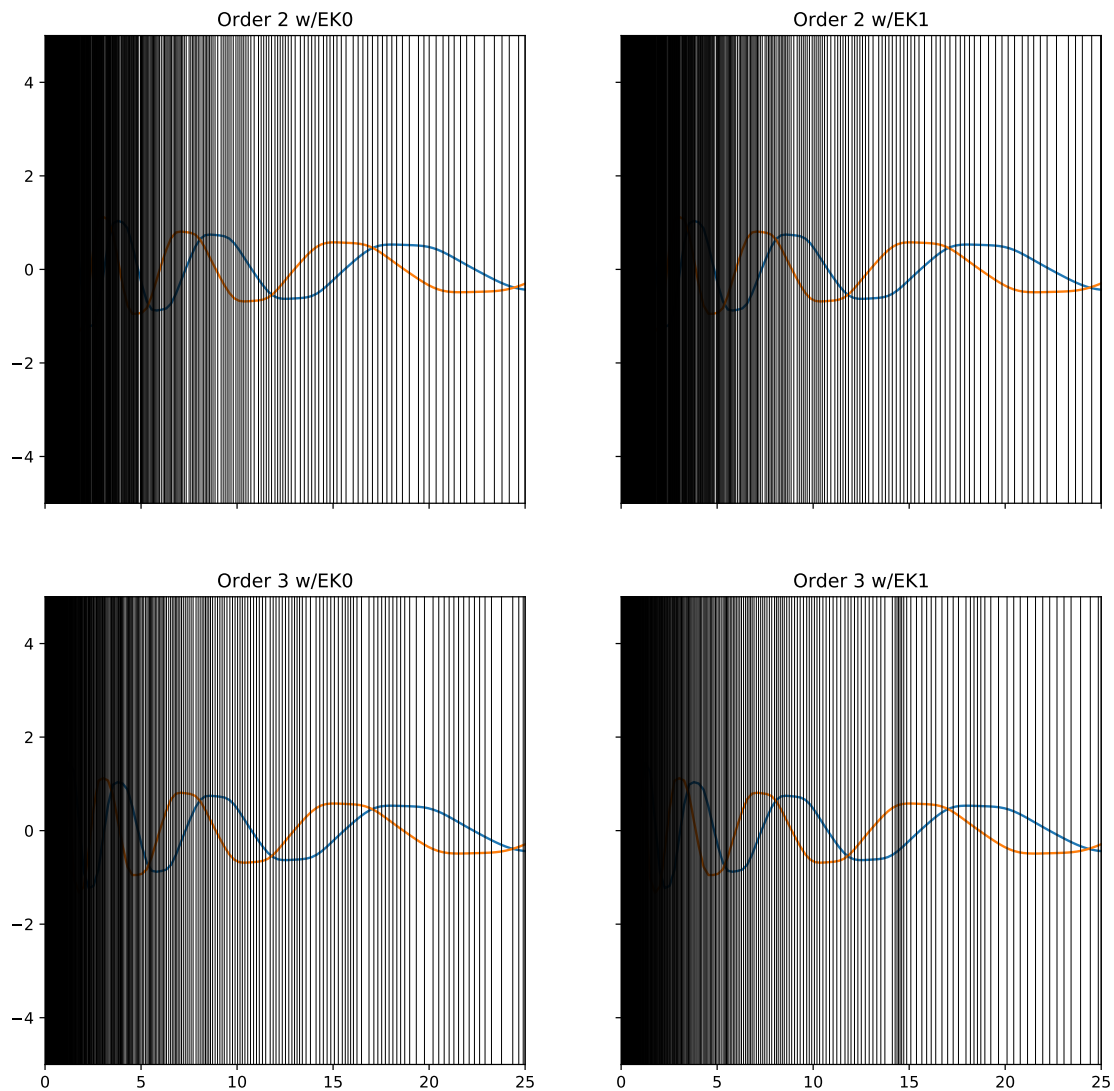
```

solution = sol(evalgrid)
ts, means, stds = evalgrid, solution.mean, solution.std
ax[idx][jdx].plot(ts, means)

for t in sol.locations:
    ax[idx][jdx].axvline(t, linewidth=0.2, color="black")
ax[idx][jdx].set_title(f"Order {algo_orders[idx]} w/{filters[jdx]}")
ax[idx][jdx].set_xlim((t0, tmax))
ax[idx][jdx].set_ylim((-5, 5))

plt.show()

```



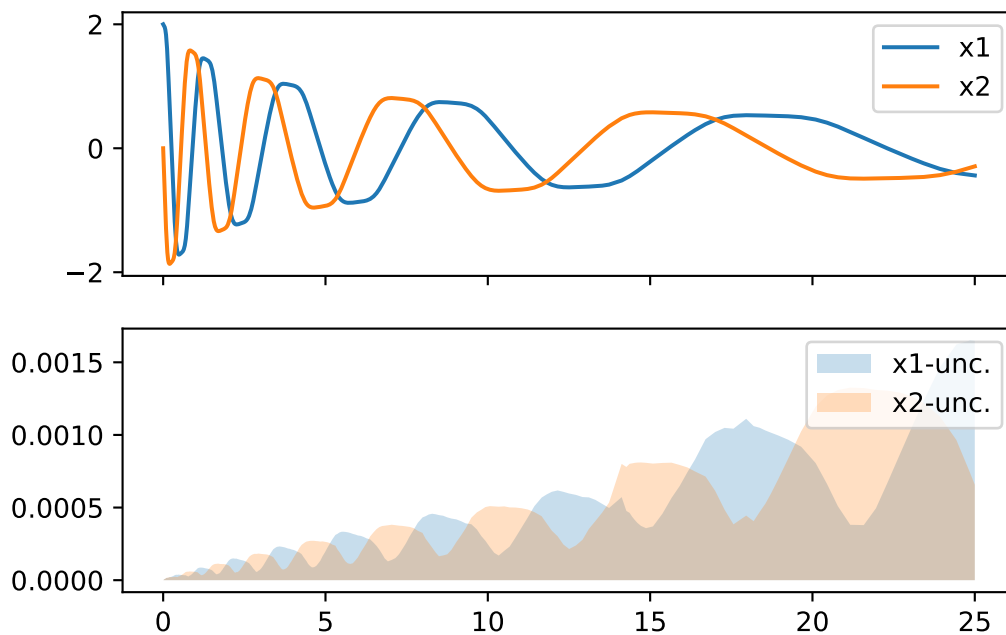
```
[28]: """
      sol = probsolve_ivp(
          f, t0, tmax, y0, df=df, step=0.1, adaptive=False, diffusion_model="dynamic"
      )
      """
```

```
[28]: '\nsol = probsolve_ivp(\n    f, t0, tmax, y0, df=df, step=0.1, adaptive=False,\n    diffusion_model="dynamic"\n)\n'
```

```
[29]: means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.legend()
fig.suptitle("EKO Solution")
plt.show()
```

EKO Solution



```
[30]: """
      sol = probsolve_ivp(
```

```

    f,
    t0,
    tmax,
    y0,
    df=df,
    step=0.5,
    algo_order=1,
    adaptive=False,
    diffusion_model="dynamic",
)
"""

```

```

[30]: '\nsol = probsolve_ivp(\n    f,\n    t0,\n    tmax,\n    y0,\n    df=df,\n    step=0.5,\n    algo_order=1,\n    adaptive=False,\n    diffusion_model="dynamic",\n)\n'

```

```

[31]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

```

```

[32]: fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[:5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,

```

```

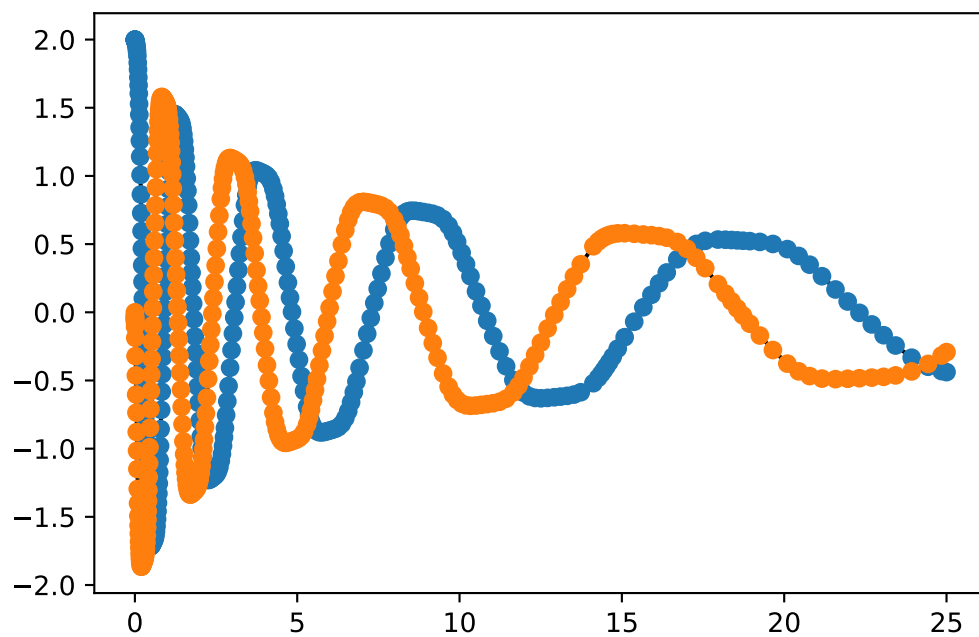
means[:, 1] - 3 * stds[:, 1],
means[:, 1] + 3 * stds[:, 1],
color="C1",
alpha=0.3,
)

ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

plt.show()

```

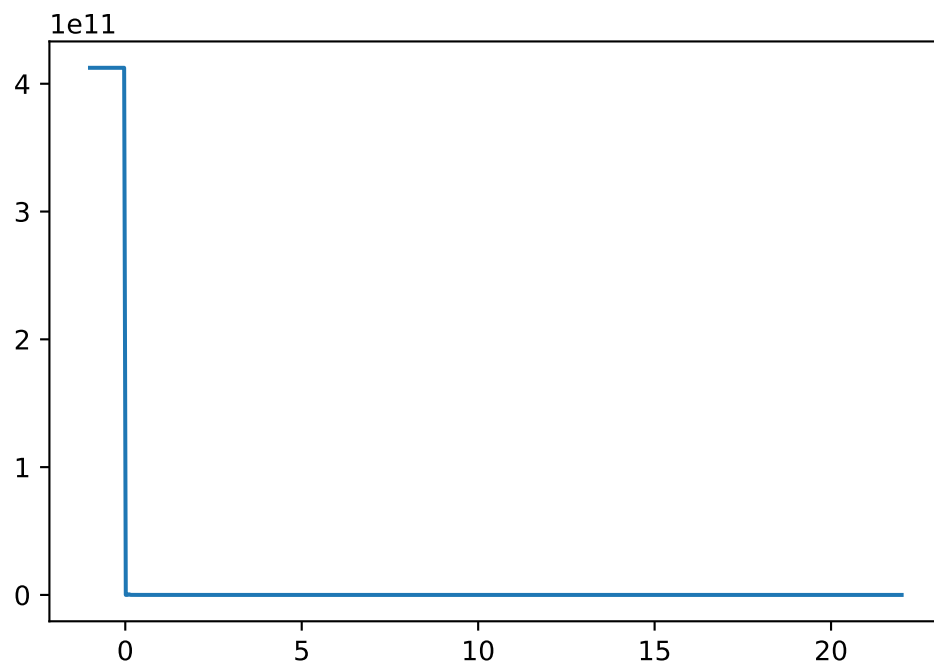
Samples From an EK0 Solution



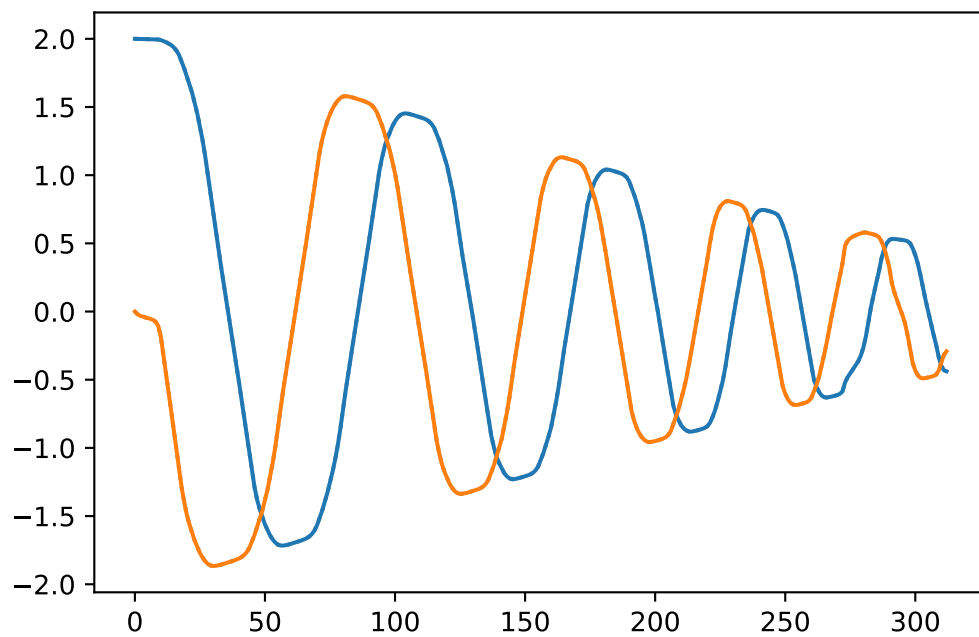
```

[33]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()

```



```
[34]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

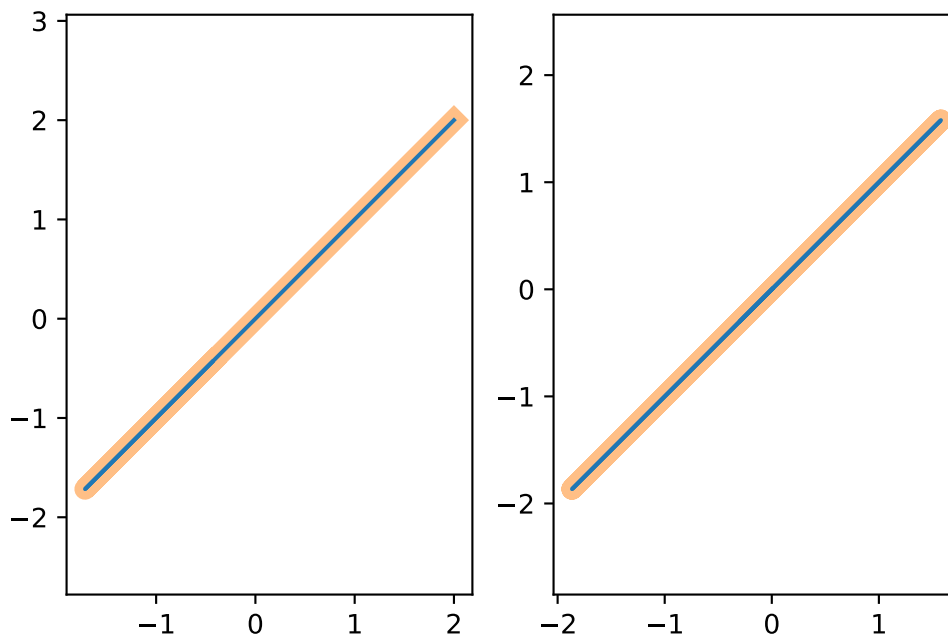


```
[35]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2) = plt.subplots(ncols=2)

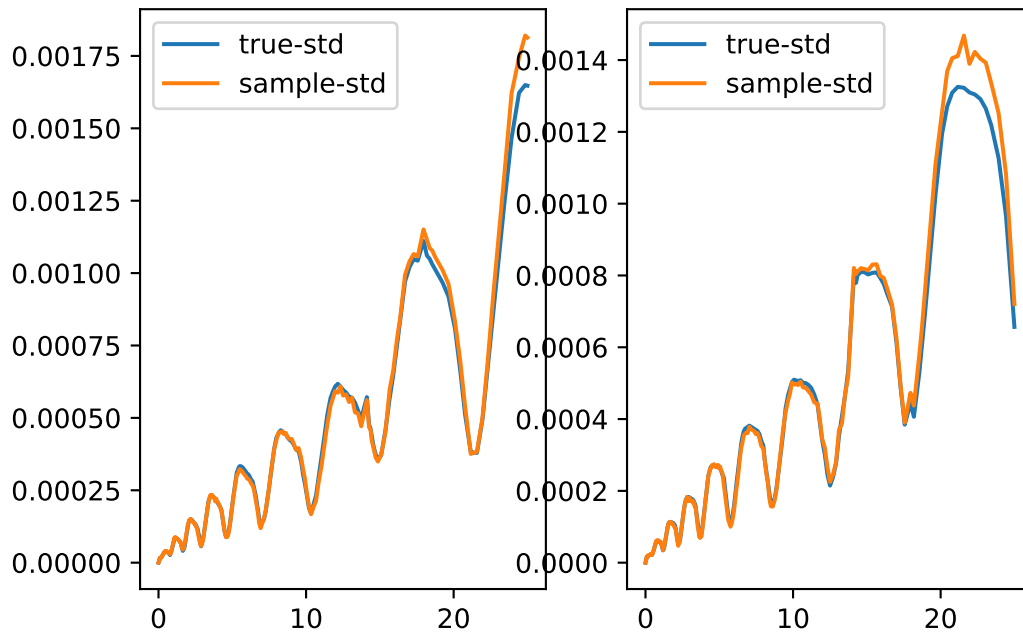
ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



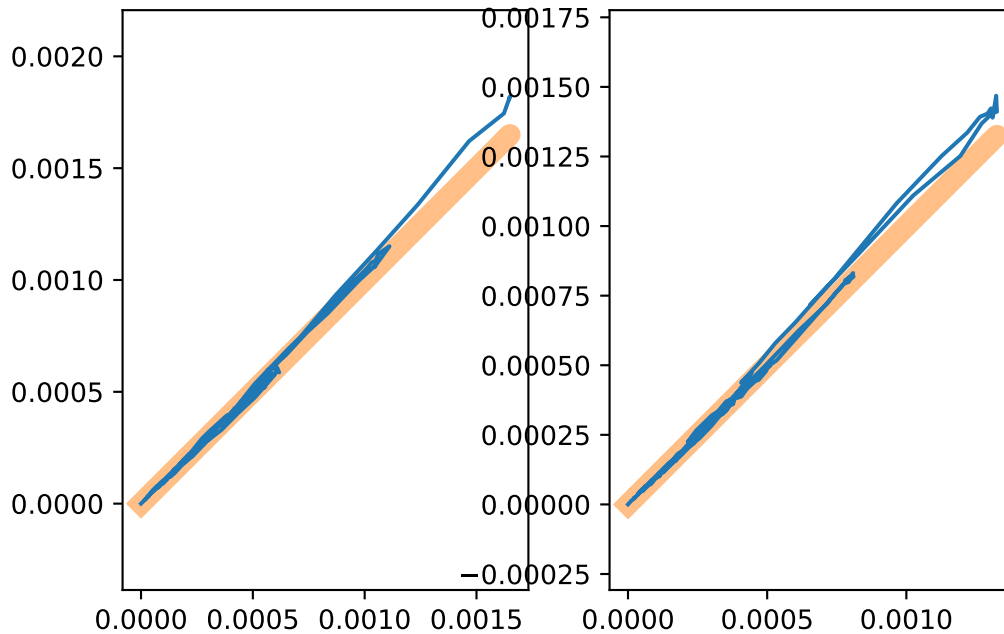
```
[36]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()
```



```
[37]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



[]:

```
[38]: from probnum import diffeq, filtsmooth, randvars, randprocs, problems
import numpy as np

import matplotlib.pyplot as plt

def f(t, y):
    y1, y2 = y
    return np.array([-0.1*y1**3+2*y2**3, -2*y1**3-0.1*y2**3])

def df(t, y):
    y1, y2 = y
    return np.array([[-0.3*y1**2, 2*y2**2], [-2*y1**2, -0.1*y2**2]])

t0 = 0.0
tmax = 25.0
y0 = np.array([2, 0])
ivp = problems.InitialValueProblem(t0=t0, tmax=tmax, y0=y0, f=f, df=df)

prior_process = randprocs.markov.integrator.IntegratedWienerProcess(
    initarg=ivp.t0,
    num_derivatives=4,
```

```

    wiener_process_dimension=ivp.dimension,
    forward_implementation="sqrt",
    backward_implementation="sqrt",
)

```

```

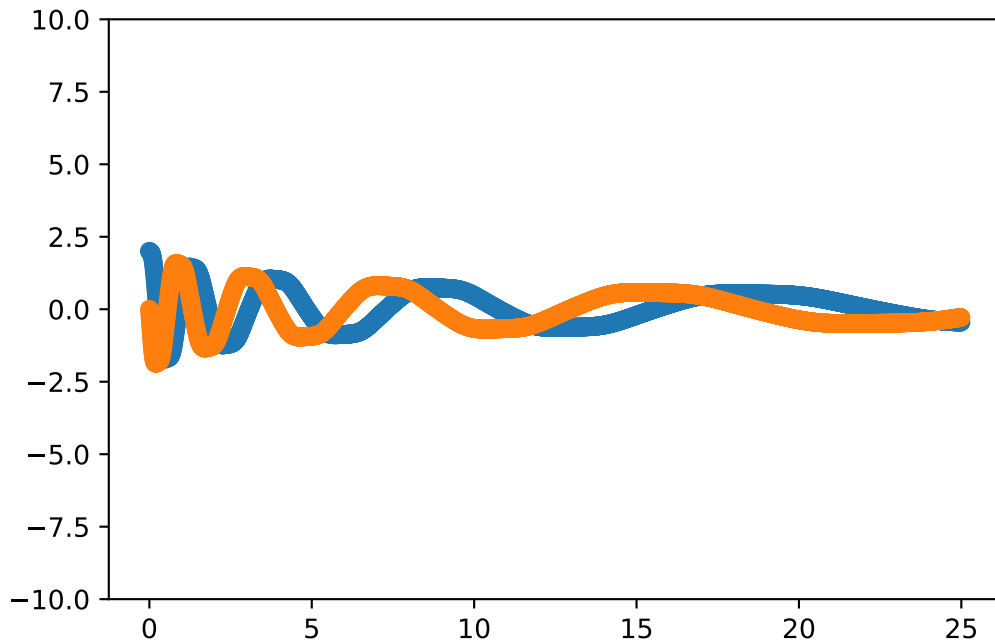
[39]: diffmodel = randprocs.markov.continuous.PiecewiseConstantDiffusion(t0=t0)
rk_init = diffeq.odefilter.initialization_routines.RungeKuttaInitialization()
ode_residual = diffeq.odefilter.information_operators.ODEResidual(4, ivp.
    ↪dimension)
ek1 = diffeq.odefilter.approx_strategies.EK1()
firststep = diffeq.stepsize.propose_firststep(ivp)
steprule = diffeq.stepsize.AdaptiveSteps(firststep=firststep, atol=1e-3, ↪
    ↪rtol=1e-5)

solver = diffeq.odefilter.ODEFilter(
    steprule,
    prior_process=prior_process,
    information_operator=ode_residual,
    approx_strategy=ek1,
    initialization_routine=rk_init,
    diffusion_model=diffmodel,
    with_smoothing=True,
)

odesol = solver.solve(ivp=ivp)
evalgrid = np.arange(ivp.t0, ivp.tmax, step=0.01)
sol = odesol(evalgrid)

plt.plot(evalgrid, sol.mean, "o-", linewidth=1)
plt.ylim((-10, 10))
plt.show()

```



3 Brusselator

```
[40]: def f(t, y):
        y1, y2 = y
        return np.array([1-4*y1+y1**2*y2, 3*y1-y1**2*y2])

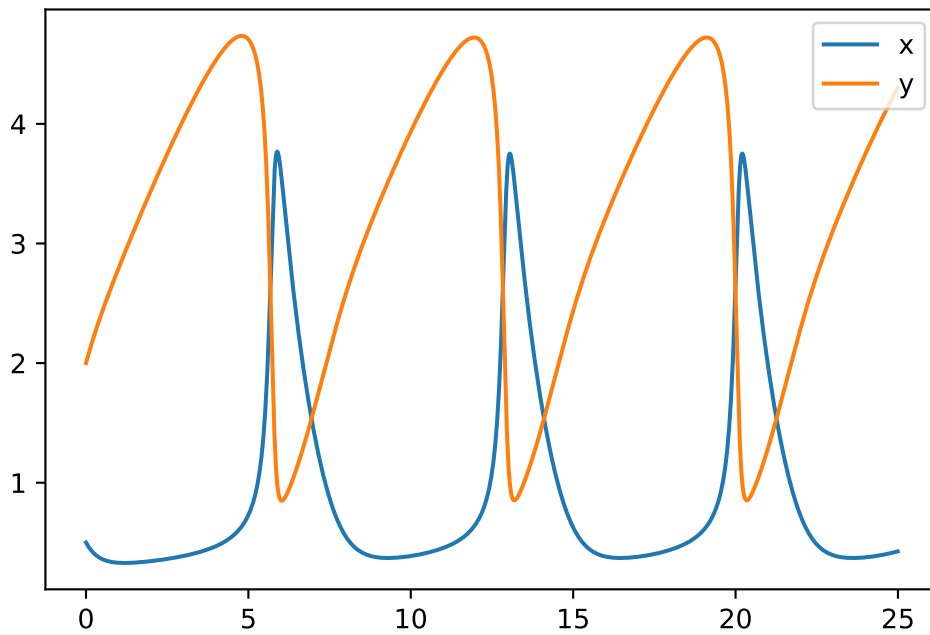
    def df(t, y):
        y1, y2 = y
        return np.array([[-4+2*y1*y2, y1**2], [3-2*y1*y2, -y1**2]])

    t0 = 0.0
    tmax = 25.0
    y0 = np.array([0.5, 2])
```

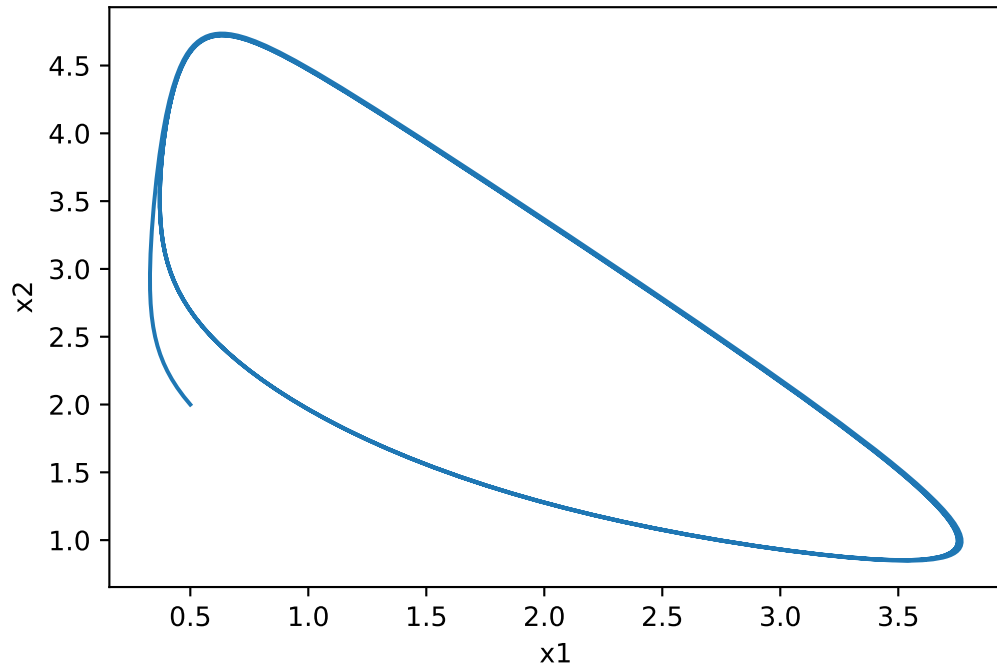
```
[41]: # IBM(1) with EK0, solving an IVP with adaptive steps works.
    sol = probsolve_ivp(f, t0, tmax, y0, df=df, algo_order=4, method="EK1")
    dt=0.01
    evalgrid = np.arange(0, 25, dt)
    means = sol(evalgrid).mean

    plt.plot(evalgrid, means[:, 0], label="x")
```

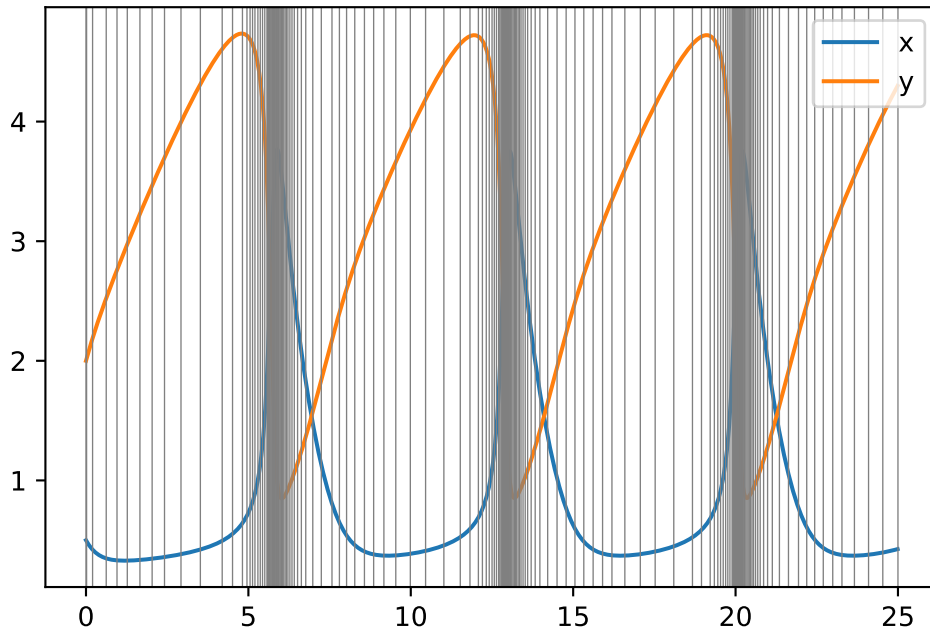
```
plt.plot(evalgrid, means[:, 1], label="y")
plt.legend(loc="upper right")
plt.show()
```



```
[42]: plt.plot(means[:, 0], means[:, 1])
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()
```



```
[43]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```



```
[44]: algo_orders = [2, 3]
filters = ["EK0", "EK1"]

fig, ax = plt.subplots(
    nrows=len(algo_orders),
    ncols=len(filters),
    sharex=True,
    sharey=True,
    figsize=(12, 12),
)

evalgrid = np.linspace(t0, tmax, 100)

for idx in range(len(algo_orders)): # algo_orders in rows
    for jdx in range(len(filters)): # filters in cols
        sol = probsolve_ivp(
            f,
            t0,
            tmax,
            y0,
            df=df,
            algo_order=algo_orders[idx],
            method=filters[jdx],
        )
```

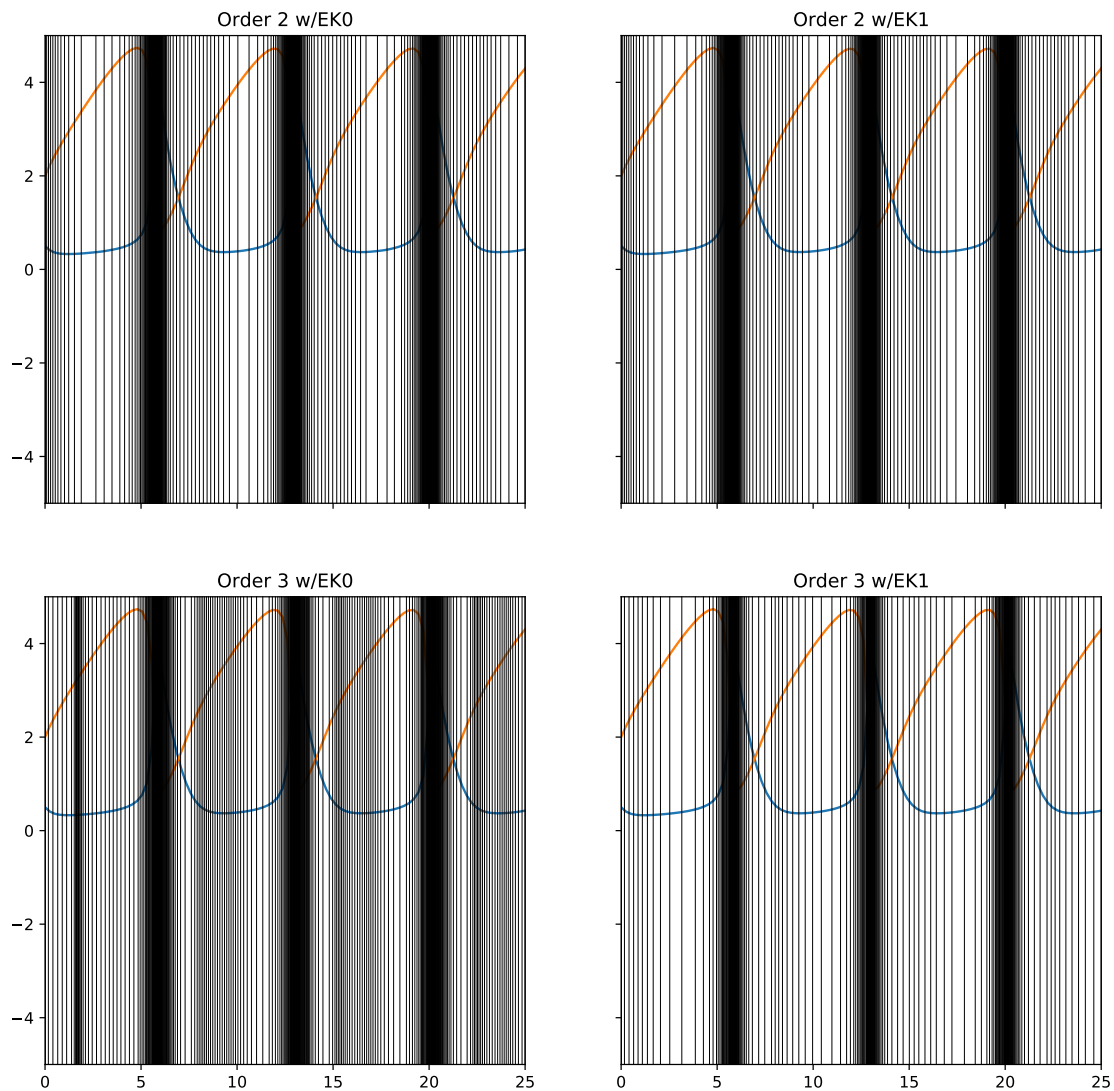
```

solution = sol(evalgrid)
ts, means, stds = evalgrid, solution.mean, solution.std
ax[idx][jdx].plot(ts, means)

for t in sol.locations:
    ax[idx][jdx].axvline(t, linewidth=0.2, color="black")
ax[idx][jdx].set_title(f"Order {algo_orders[idx]} w/{filters[jdx]}")
ax[idx][jdx].set_xlim((t0, tmax))
ax[idx][jdx].set_ylim((-5, 5))

plt.show()

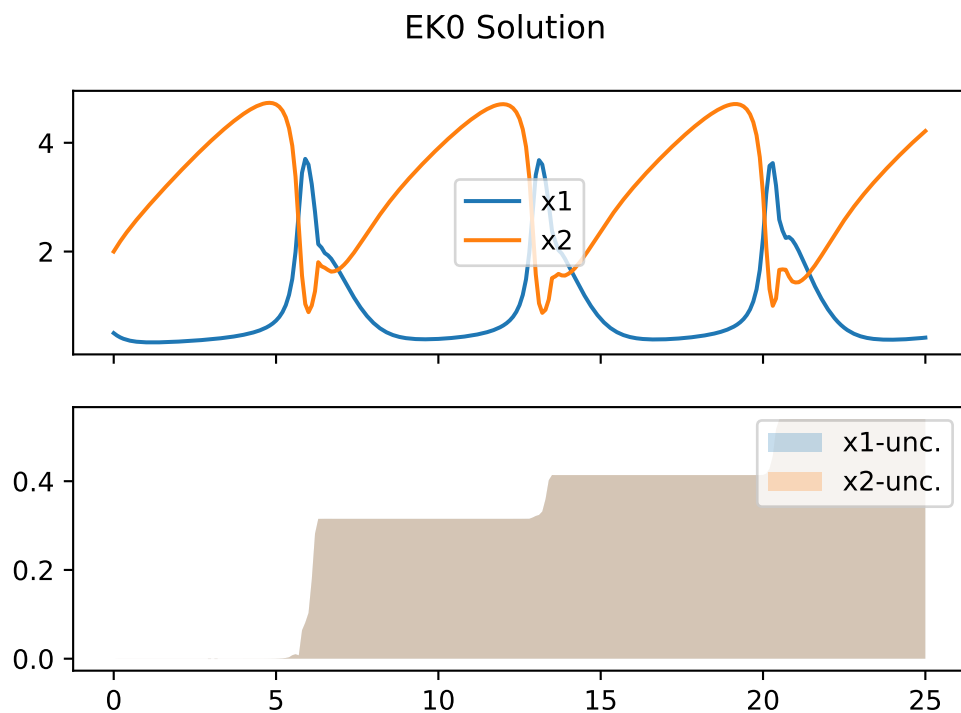
```



```
[45]: sol = probsolve_ivp(
        f, t0, tmax, y0, df=df, step=0.1, adaptive=False, diffusion_model="dynamic"
    )

means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.legend()
fig.suptitle("EK0 Solution")
plt.show()
```



```
[46]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)
```

```

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

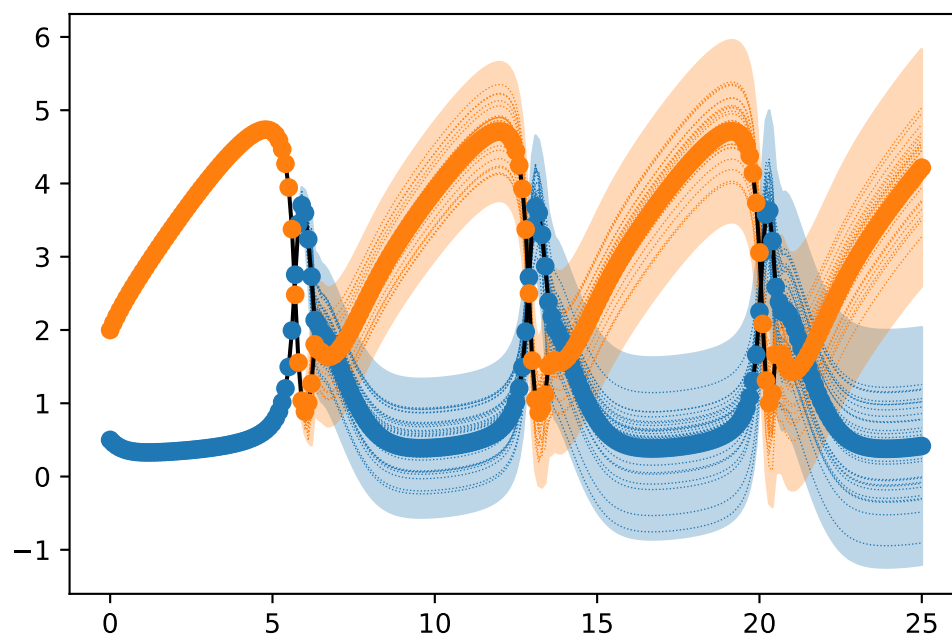
fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[::5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)

ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

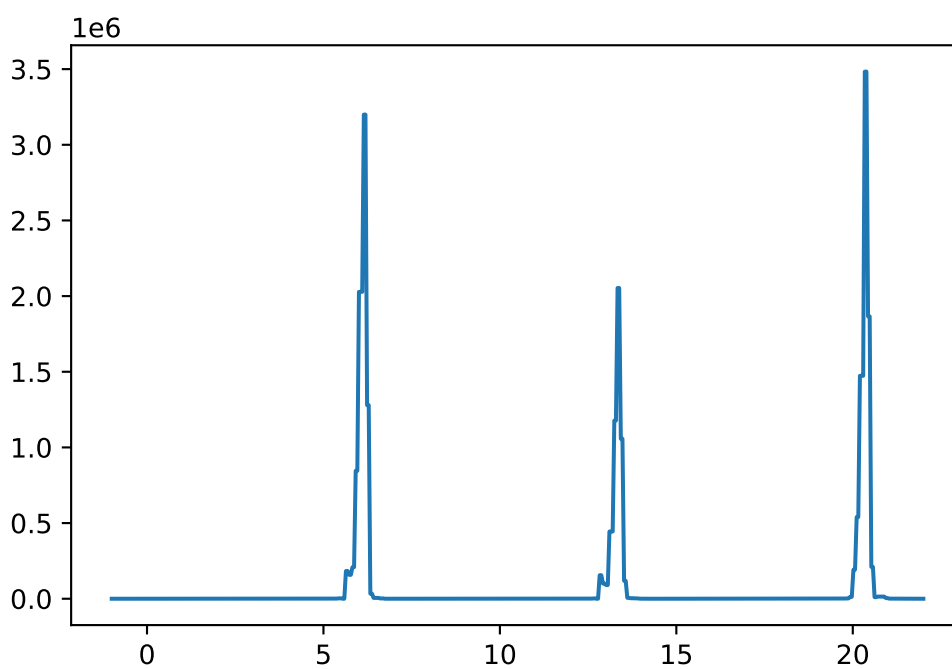
plt.show()

```

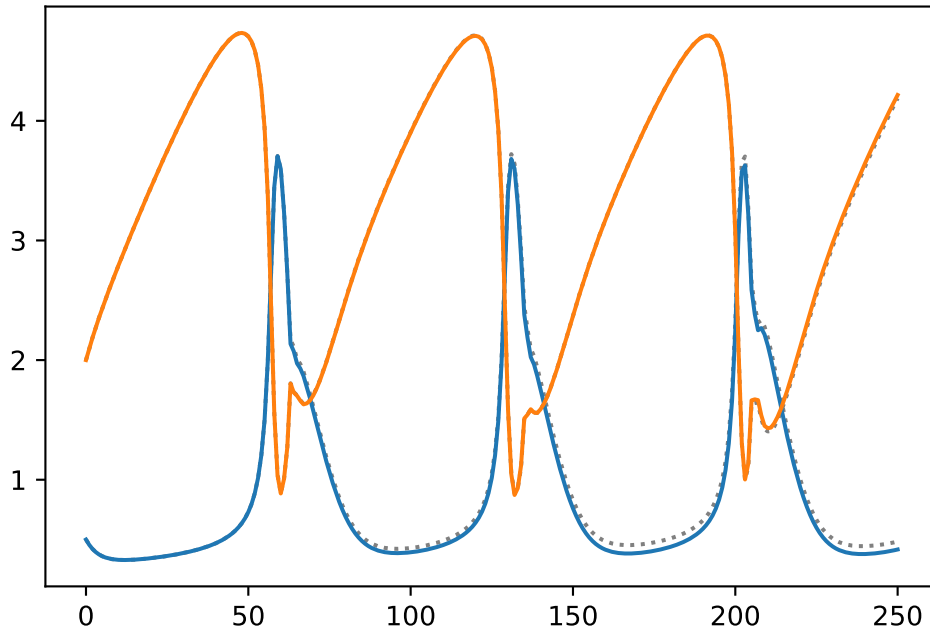
Samples From an EK0 Solution



```
[47]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[48]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

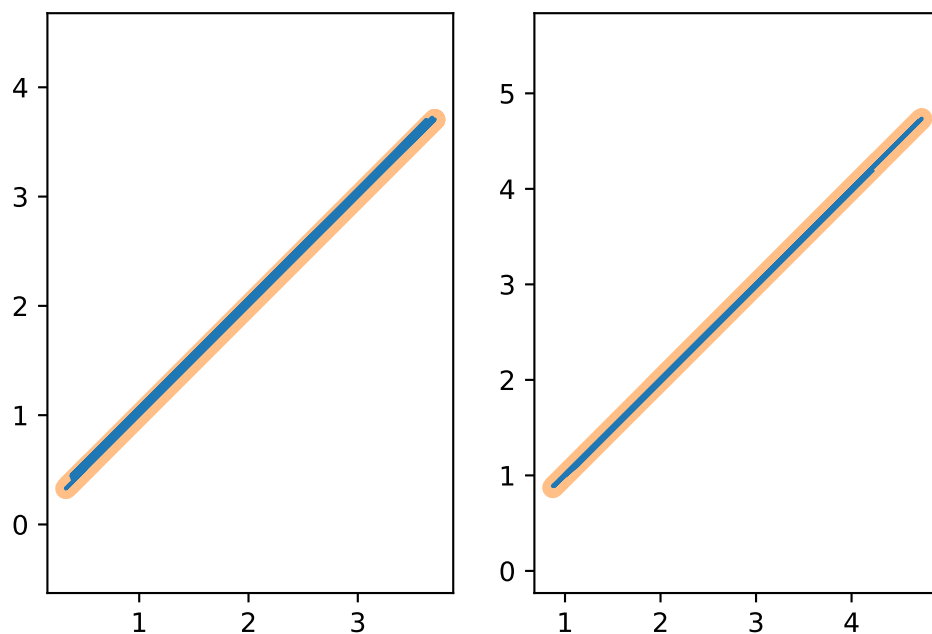


```
[49]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2) = plt.subplots(ncols=2)

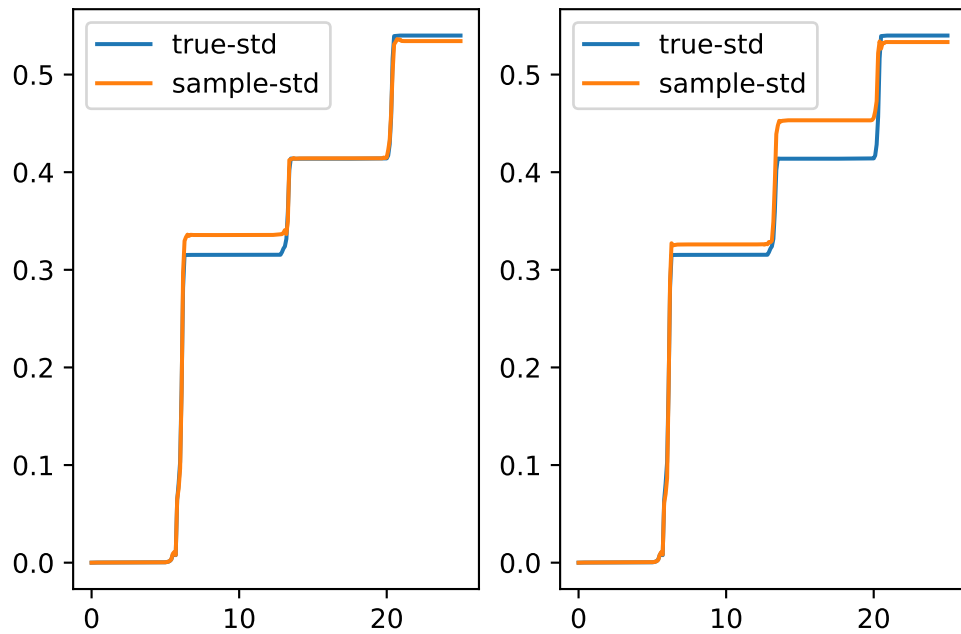
ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



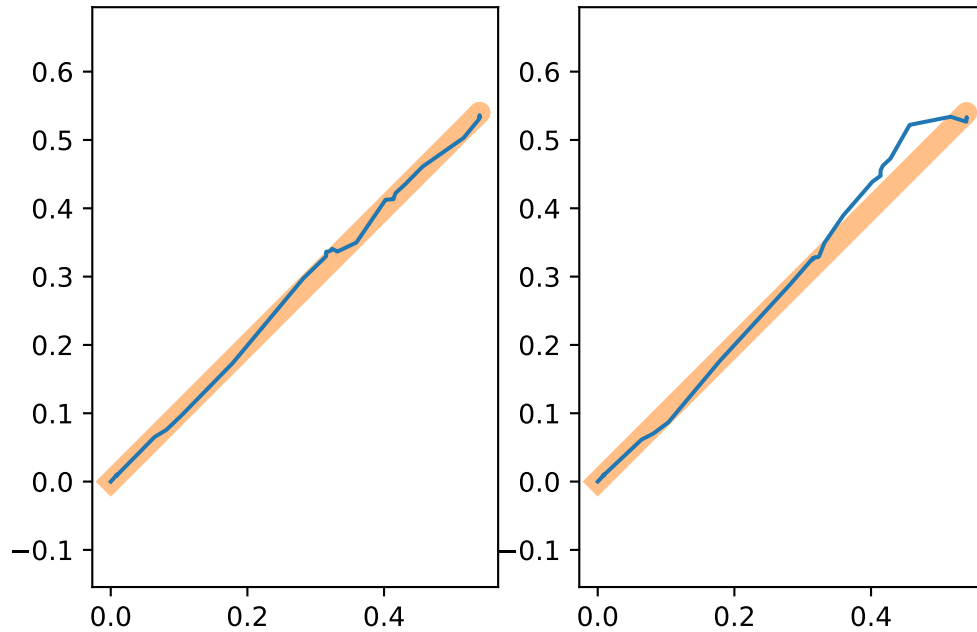
```
[50]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()
```



```
[51]: fig, (ax1, ax2) = plt.subplots(ncols=2)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
plt.show()
```



[]:

[]:

4 Three-dimensional linear system

```
[57]: def f(t, y):
      y1, y2, y3 = y
      return np.array([-0.1*y1-2*y2, 2*y1-0.1*y2, -0.3*y3])

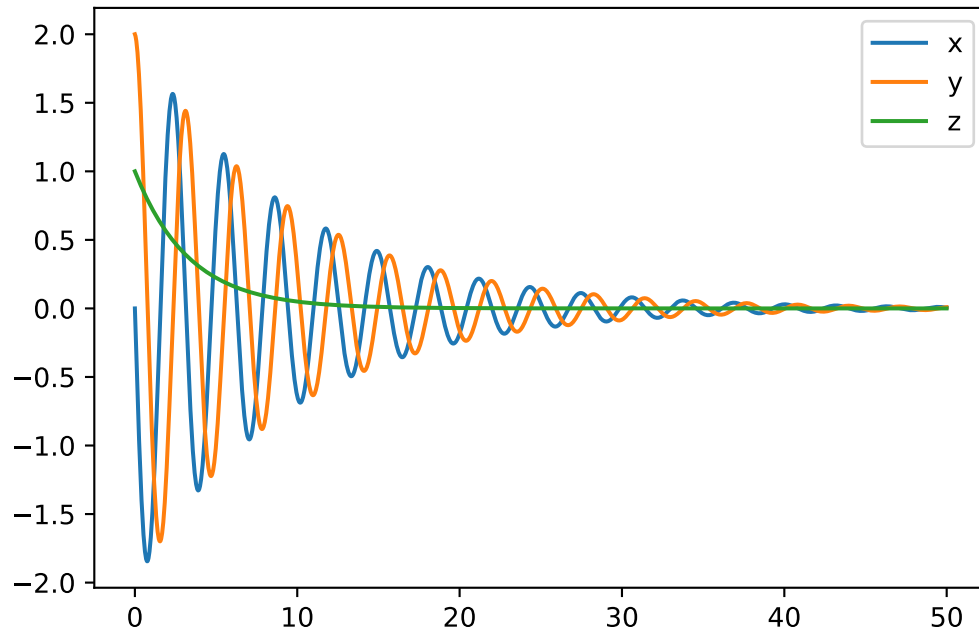
      def df(t, y):
          y1, y2, y3 = y
          return np.array([[-0.1, 2, 0],[2, -0.1, 0],[0,0,-0.3]])
```

```
[58]: t0 = 0.0
      tmax = 50.0
      y0 = np.array([0,2,1])
```

```
[59]: # IBM(1) with EK0, solving an IVP with adaptive steps works.
      sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.1, adaptive=False,
      ↪diffusion_model="dynamic")
      dt=0.01
```

```
evalgrid = np.arange(0,50, dt)
means = sol(evalgrid).mean
```

```
[60]: plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
plt.legend(loc="upper right")
plt.show()
```



```
[62]: from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(4, 4))
ax = fig.add_subplot(1, 1, 1, projection="3d")
ax.plot(
    means[:, 0],
    means[:, 1],
    means[:, 2]
)
ax.set(title='3D plot', xlabel="$x$", ylabel="$y$", zlabel="$z$")
ax.legend()
fig.show()
```

No handles with labels found to put in legend.

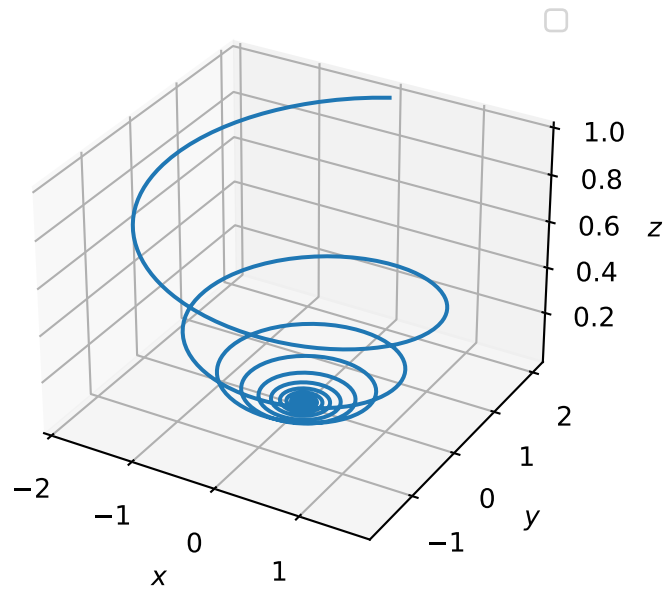
/home/kkumar/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:11:

UserWarning: Matplotlib is currently using

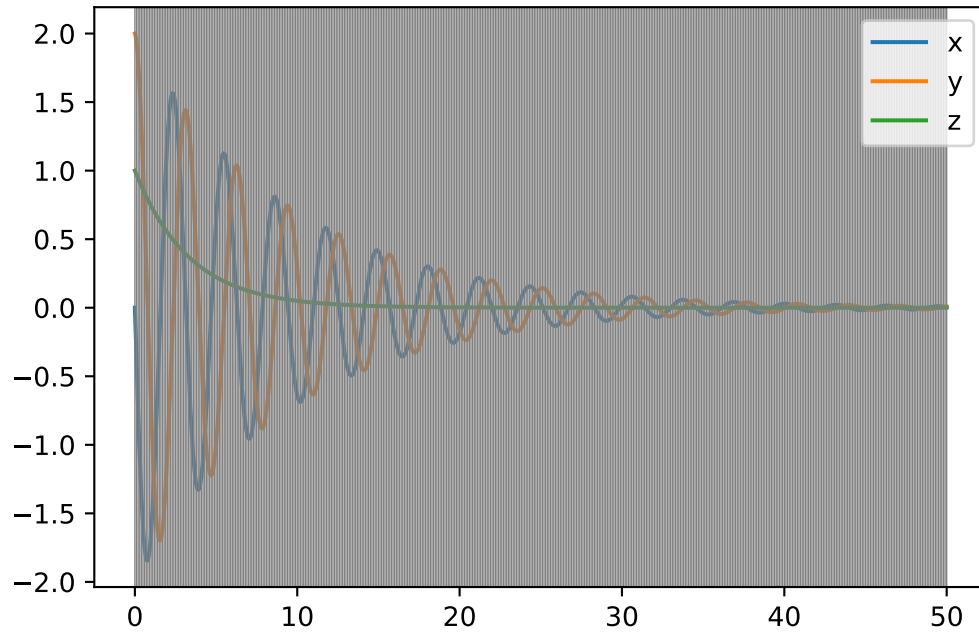
module://ipykernel pylab.backend_inline, which is a non-GUI backend, so cannot show the figure.

```
# This is added back by InteractiveShellApp.init_path()
```

Training data



```
[63]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```

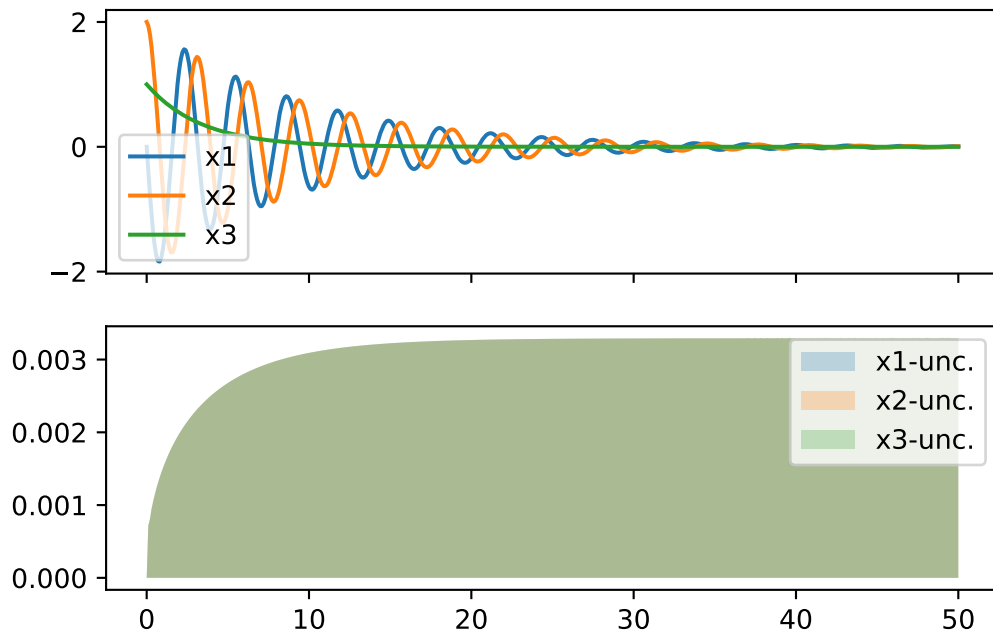


```
[65]: sol = probsolve_ivp(
    f, t0, tmax, y0, df=df, step=0.1, adaptive=False, diffusion_model="dynamic"
)

means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.plot(sol.locations, means[:, 2], label="x3")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.fill_between(sol.locations, stds[:, 2], alpha=0.25, label="x3-unc.")
ax2.legend()
fig.suptitle("EKO Solution")
plt.show()
```

EK0 Solution



```
[66]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[:5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 2], ":", color="C2", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
```

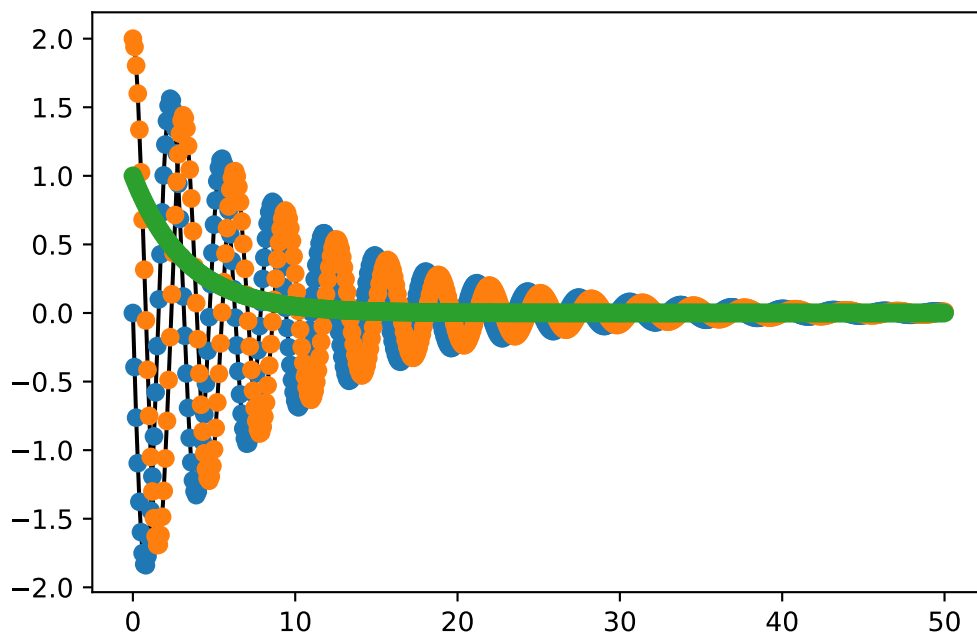
```

ax1.plot(locations, means[:, 2], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 2] - 3 * stds[:, 2],
    means[:, 2] + 3 * stds[:, 2],
    color="C2",
    alpha=0.3,
)
ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

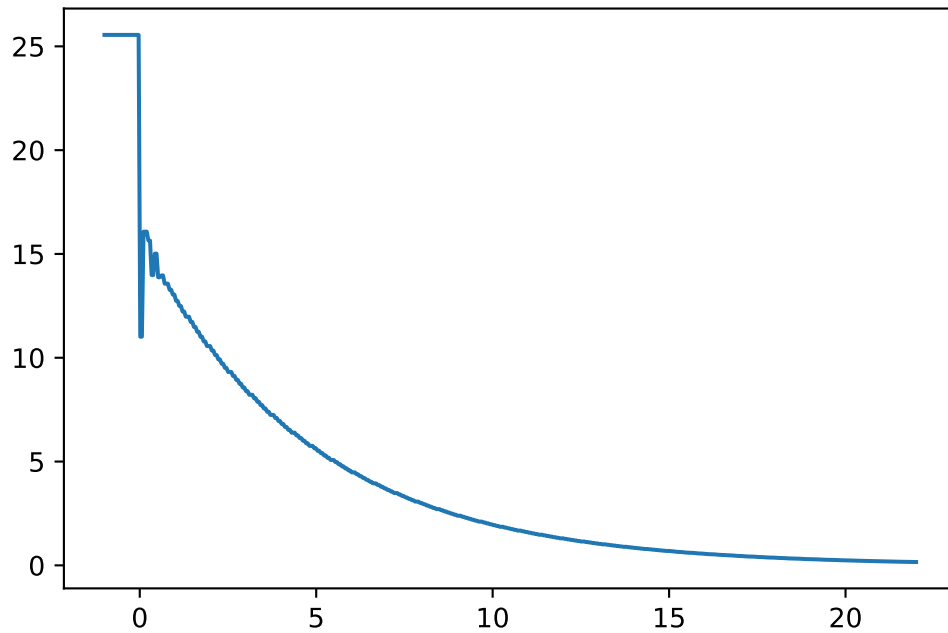
plt.show()

```

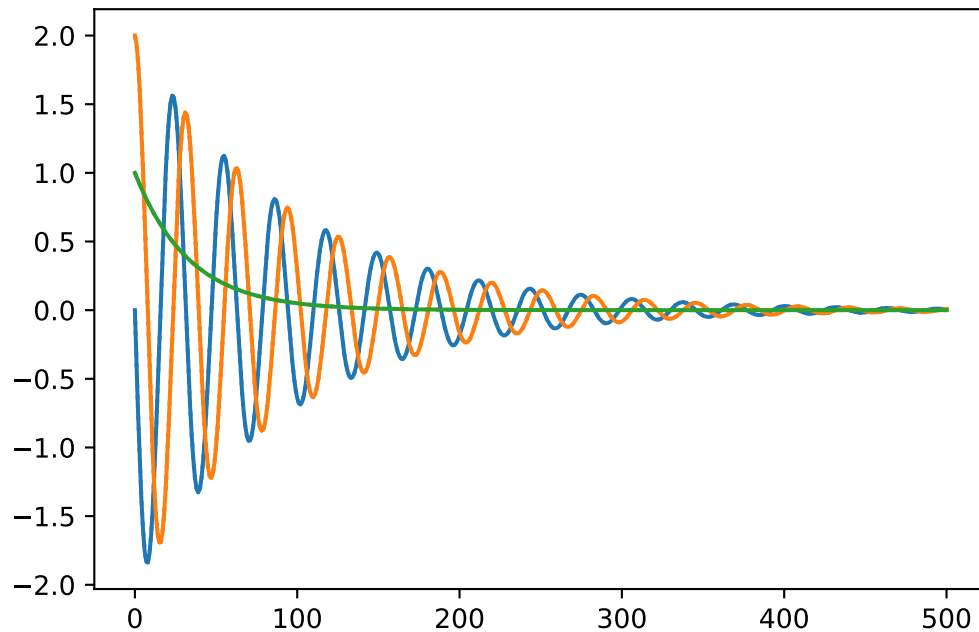
Samples From an EK0 Solution



```
[67]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[68]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

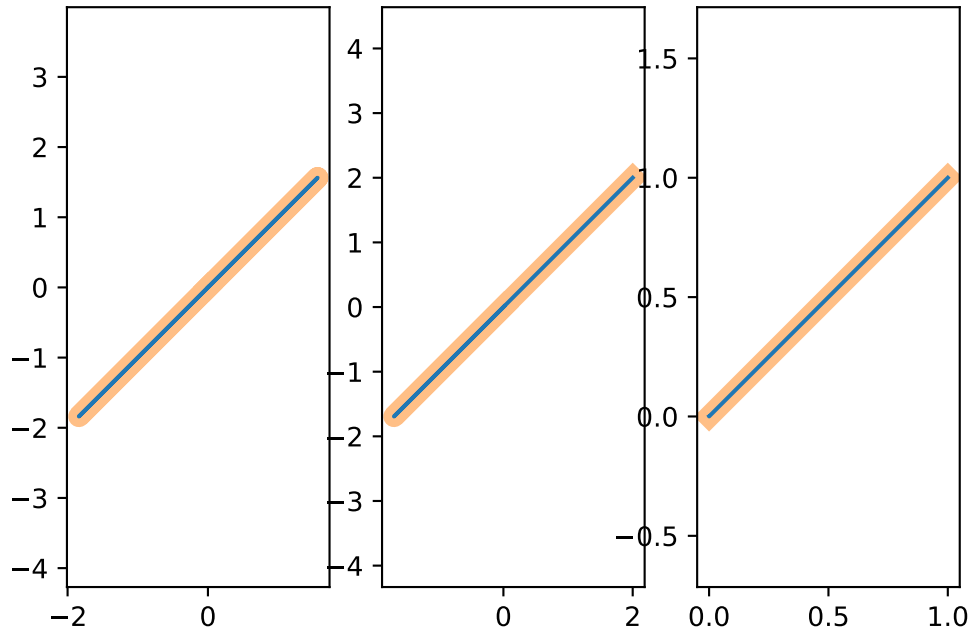


```
[69]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax3.plot(means[:, 2], sample_mean[:, 2])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(means[:, 2], means[:, 2], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```

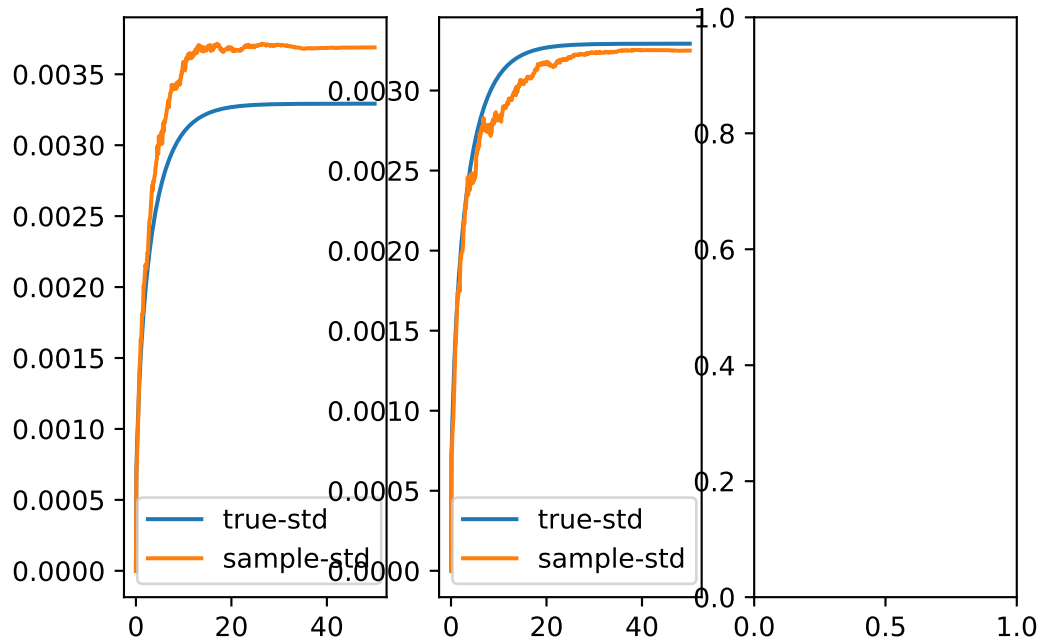


```
[71]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()

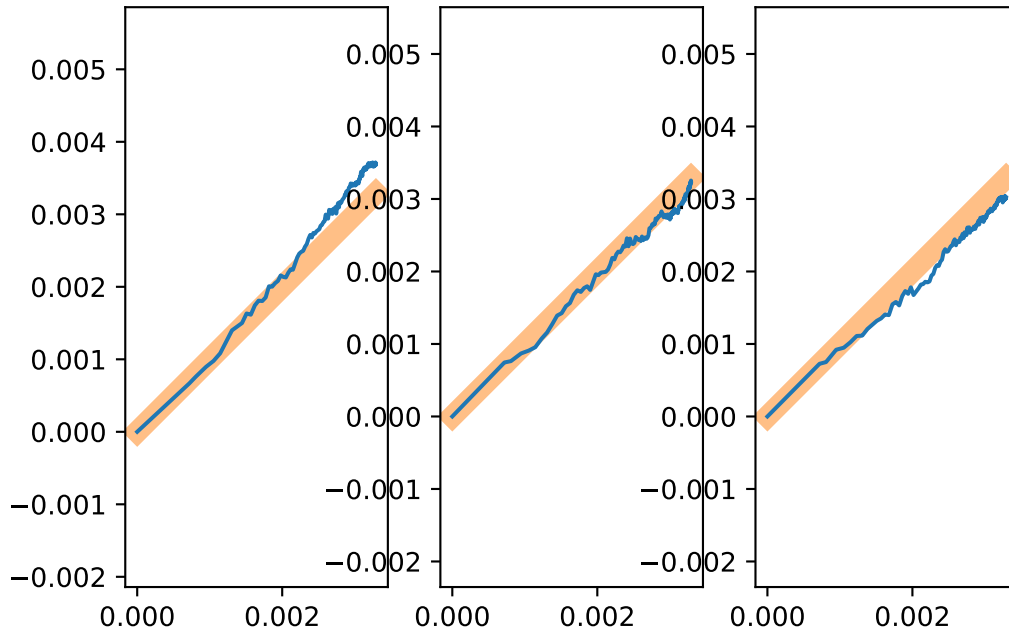
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()

ax3.plot(locations, stds[:, 2], label="true-std")
ax3.plot(locations, sample_std[:, 2], label="sample-std")
ax3.legend()
plt.show()
```



```
[72]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax3.plot(stds[:, 2], sample_std[:, 2])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(stds[:, 2], stds[:, 2], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```



[]:

5 Rossler Systems

```
[73]: def f(t, y):
        y1, y2, y3 = y
        return np.array([-y2-y3, y1+0.2*y2, 0.2+y3*(y1-5.7)])

    def df(t, y):
        y1, y2, y3 = y
        return np.array([[0, -1, -1], [1, 0.2, 0], [y3, 0, y1-5.7]])
```

```
[74]: t0 = 0.0
        tmax = 25.0
        y0 = np.array([0,0,0])
```

```
[77]: # IBM(1) with EK0, solving an IVP with adaptive steps works.
        sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.1, adaptive=False,
        ↪diffusion_model="dynamic")
        dt=0.01
        evalgrid = np.arange(0,25, dt)
        means = sol(evalgrid).mean
```

```

-----
ValueError                                Traceback (most recent call last)
<ipython-input-77-87fcc8647e5> in <module>
      1 # IBM(1) with EK0, solving an IVP with adaptive steps works.
----> 2 sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.1, adaptive=False,
    ↪ diffusion_model="dynamic")
      3 dt=0.01
      4 evalgrid = np.arange(0,25, dt)
      5 means = sol(evalgrid).mean

~/anaconda3/lib/python3.7/site-packages/probnum/diffeq/_probsolve_ivp.py in
    ↪ probsolve_ivp(f, t0, tmax, y0, df, method, dense_output, algo_order, adaptive
    ↪ atol, rtol, step, diffusion_model, time_stops)
      297     )
      298
--> 299     return solver.solve(ivp=ivp, stop_at=time_stops)

~/anaconda3/lib/python3.7/site-packages/probnum/diffeq/_odesolver.py in
    ↪ solve(self, ivp, stop_at, callbacks)
      45         """
      46         times, rvs = [], []
----> 47         for state in self.solution_generator(ivp, stop_at=stop_at,
    ↪ callbacks=callbacks):
      48             times.append(state.t)
      49             rvs.append(state.rv)

~/anaconda3/lib/python3.7/site-packages/probnum/diffeq/_odesolver.py in
    ↪ solution_generator(self, ivp, stop_at, callbacks)
      72             dt = time_stopper.adjust_dt_to_time_stops(state.t, dt)
      73
----> 74             state, dt = self.perform_full_step(state, dt)
      75
      76             if callbacks is not None:

~/anaconda3/lib/python3.7/site-packages/probnum/diffeq/_odesolver.py in
    ↪ perform_full_step(self, state, initial_dt)
      106         proposed_state = None
      107         while not step_is_sufficiently_small:
--> 108             proposed_state = self.attempt_step(state, dt)
      109
      110             # Acceptance/Rejection due to the step-rule

~/anaconda3/lib/python3.7/site-packages/probnum/diffeq/odefilter/_odefilter.py
    ↪ in attempt_step(self, state, dt)
      297         # Gain needs manual catching up, too. Use it to compute the
    ↪ update
      298         crosscov = full_pred_cov @ H.T

```

```

--> 299         gain = scipy.linalg.cho_solve((meas_rv.cov_cholesky, True),
↪crosscov.T).T
      300         zero_data = np.zeros(meas_rv.mean.shape)
      301         filt_rv, _ = self.measurement_model.backward_realization(

~/anaconda3/lib/python3.7/site-packages/scipy/linalg/decomp_cholesky.py in
↪cho_solve(c_and_lower, b, overwrite_b, check_finite)
      192         (c, lower) = c_and_lower
      193         if check_finite:
--> 194             b1 = asarray_chkfinite(b)
      195             c = asarray_chkfinite(c)
      196         else:

~/anaconda3/lib/python3.7/site-packages/numpy/lib/function_base.py in
↪asarray_chkfinite(a, dtype, order)
      487         if a.dtype.char in typecodes['AllFloat'] and not np.isfinite(a).
↪all():
      488             raise ValueError(
--> 489                 "array must not contain infs or NaNs")

      490         return a
      491

```

ValueError: array must not contain infs or NaNs

[]: